Fachhochschule Nordwestschweiz
Hochschule für Technik

Software Architecture

View Model

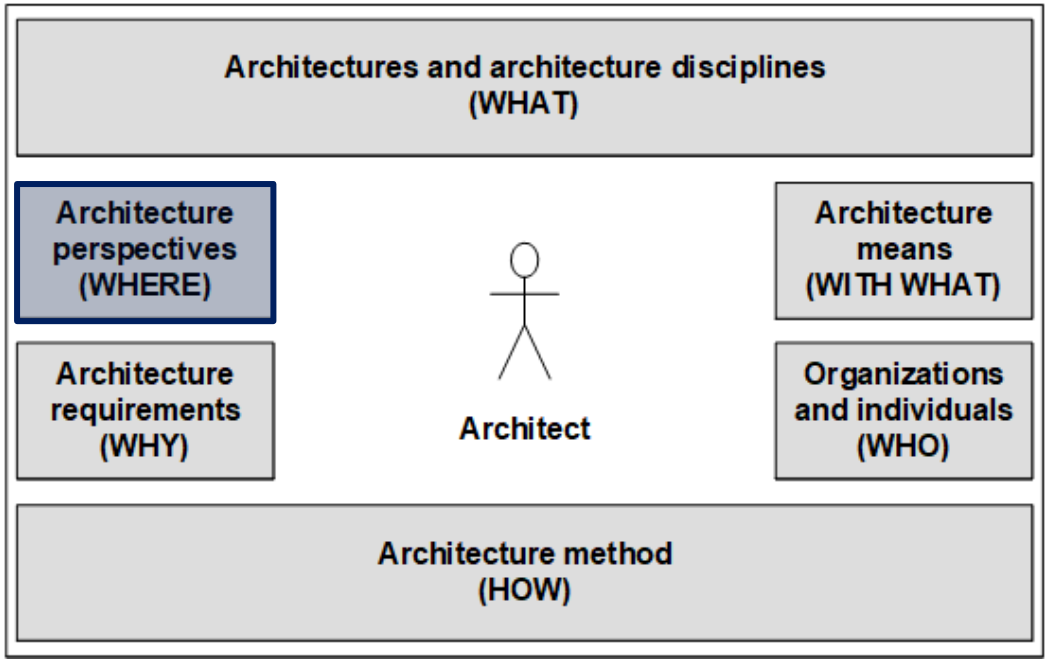
BSc

Ingo Arnold

Copyright Notice

Lecture Opening
Architecture Orientation Framework

Architecture WHERE deals with the **representation of architecture** in general as well as software architecture in particular [Vogel, Arnold et al 2011].



Lecture Opening
Motivation

In this unit, we will introduce **the concepts of architecture representation** including modelling, views, viewpoints, and view models. To do this, we will look at standard definitions and derive the core aspects of software architecture for us from these.

Beyond a general overview over view models per se, we will introduce and investigate the concrete view model of Philippe Kruchten [Kruchten 1995] .

Lecture Opening
Learning Objectives

You …

- know essential building blocks as well as the benefits of architecture view models
- understand the relationship between architecture descriptions and view models
- know that different view models exist for different insight purposes
- know Kruchten's **4+1 view model** and understand how to apply it in the context of your own architecture descriptions

Lecture Agenda

- Architecture View Model
- Kruchten 4+1 View Model
- Development View to Logical View

Architecture View Model
Modelling and Models

Modelling is probably one of the **activities** that **most people primarily associate** with **Architecture** and the profession of Architect.

A **model** is a **reduced and abstracted representation** of an **object of interest** …

... and is basically **used** to **provide targeted information about** the represented **object**.

Models hide irrelevant details in favour of the **aspects** that are **relevant** to the respective **interest**.

Models essentially **enable communication** and **collaboration** and are therefore **indispensable components** of the **Architect's toolbox**.

Architecture View Model
Modelling and Models

Modelling is probably one of the **activities** that **most people primarily associate** with **Architecture** and the profession of Architect.

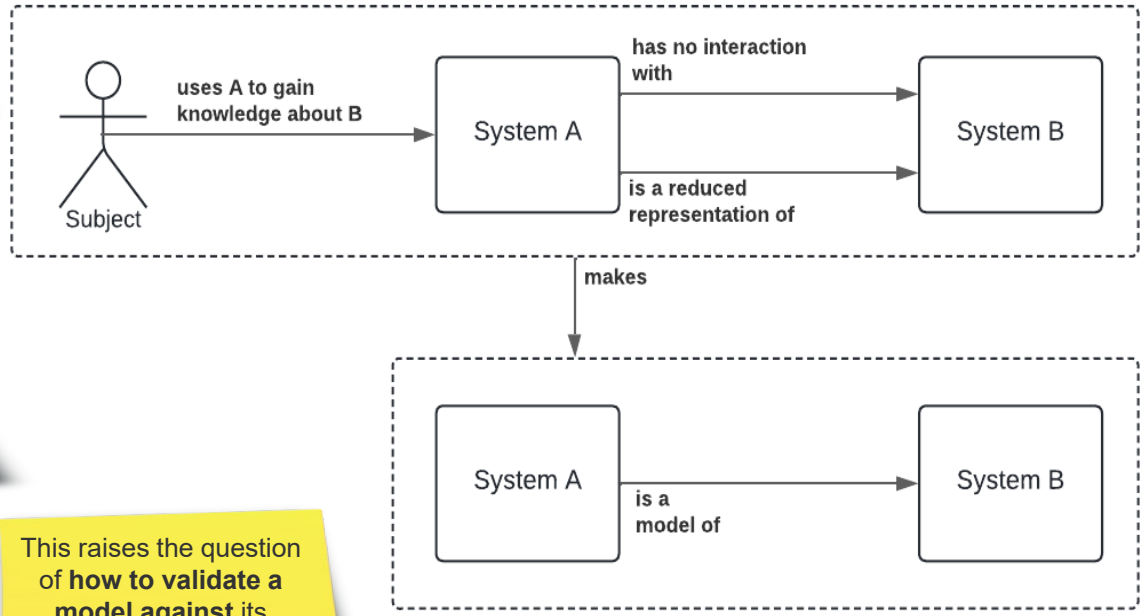
Leo Apostel ([Apostel 1960]) **gives** a very good and very fundamental **definition** of what a **Model** is.

"Any **Subject using** a **System A** that is **neither directly** nor **indirectly interacting** with **System B**, **to obtain information about** the **system B**, **is using A as a model for B**".

The **notion of Model** is almost **as important as** the **notion of System**.

Studying a System essentially **boils down** to **representing the System via a Model** and examining or **analysing its behaviour**.

This raises the question of **how to validate a model against** its **represented System**?



Architecture View Model
Modelling and Models

A **model** …

- is a **representation of a subject of interest**, where the model provides a smaller scale, and abstracted representation of the subject matter serving the interest
- **suppresses irrelevant details** in favour of those aspects relevant for the interest at hand
- needs to be **conscious of the purpose** it serves
- the effort of creating and maintaining a model must never outweigh the value of having it
- abstracts real world phenomena for the purpose of understanding aspects relevant to **answer interesting questions**
- is often based on **standardized modelling nomenclature** which can be textual, graphic, iconographic, or hybrids
- **enables communication, collaboration,** and influences how we think about and perceive the world represented by the model

Architecture View Model

Modelling and Models – Nomenclatures

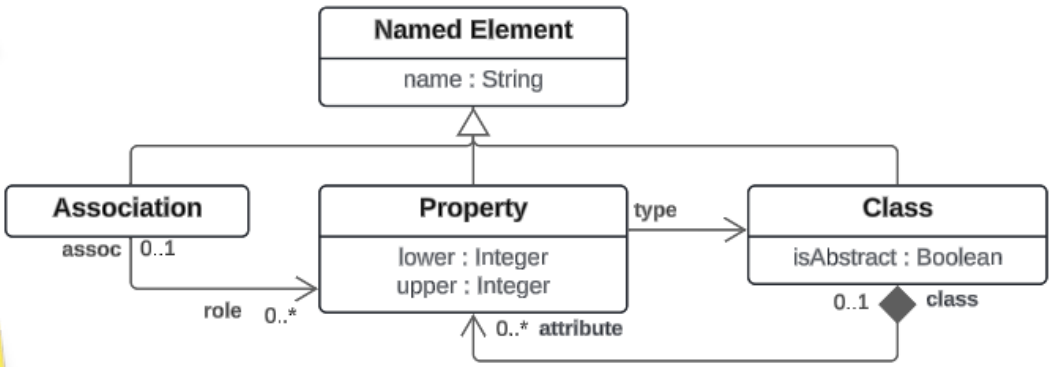
For the **formulation** of **Models**, we use (standardised and widely used) **Modelling Nomenclatures** as a **means** of **manifesting** the **Architecture**.

Many Modelling Nomenclatures exist, like the **Unified Modelling Language** (**UML**).

Modelling Languages strive to **minimise ambiguity** and at the same time be **expressive** in the respective **target domain**.

The **influence of Models** on the **way we think**, **communicate** and **perceive real-world phenomena cannot be overestimated**.

Hence the **adequacy** of the **chosen modelling language** must **not be underestimated** either.



Architecture View Model

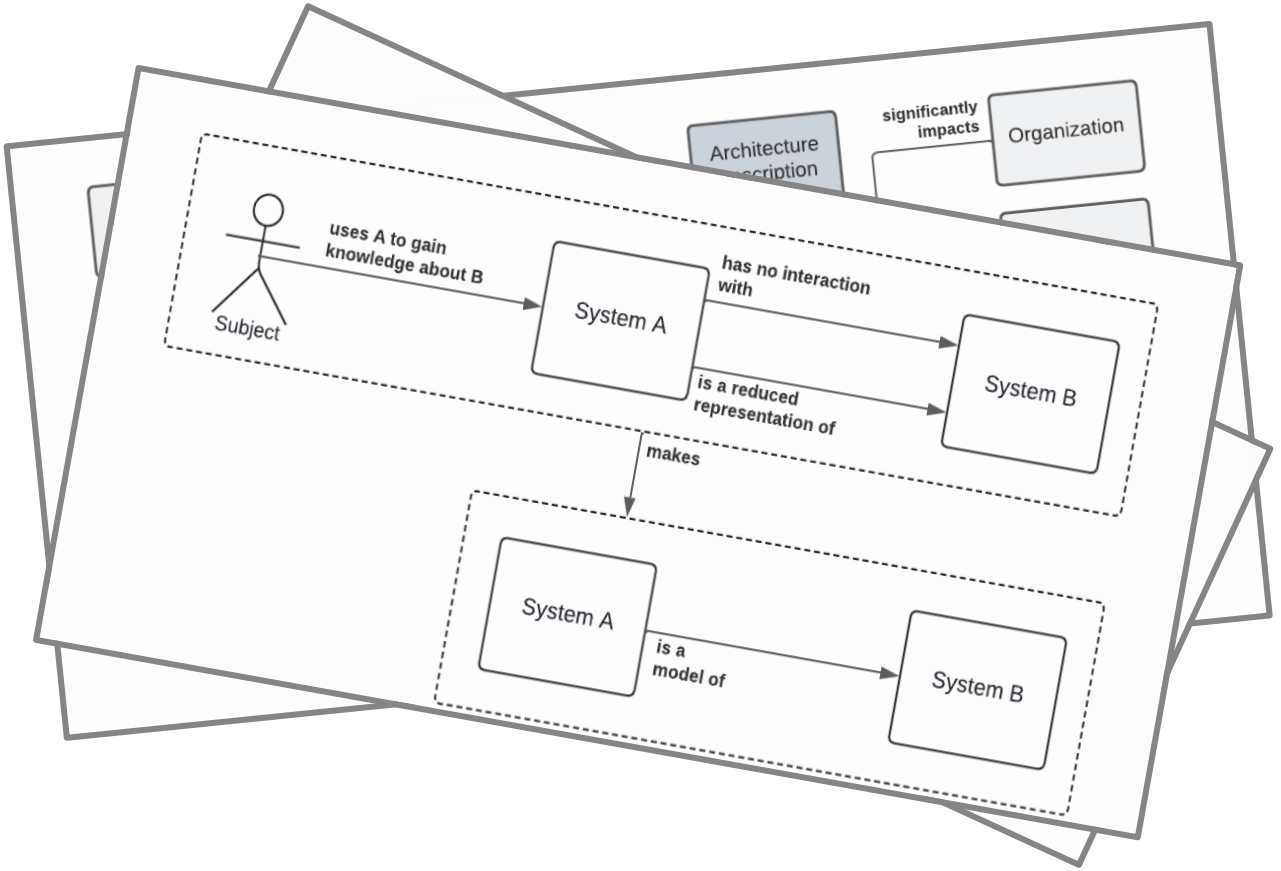
Modelling and Models – Nomenclatures

For the **formulation** of **Models**, we use (standardised and widely used) **Modelling Nomenclatures** as a **means** of **manifesting** the **Architecture**.

For example, we saw a **Model** that **captured key concepts** in the **Architecture Domain** (→ **Domain Modelling Language**).

We also saw a **Model** in which we illustrated **different types of complexities** in a **scale consisting of two dimensions** (→ **Complexity Differentiation Language**).

We even saw a **Model** in which the **concept of Model** was illustrated (→ **Model Meta-Language**).



Architecture View Model

Modelling and Models – Nomenclatures

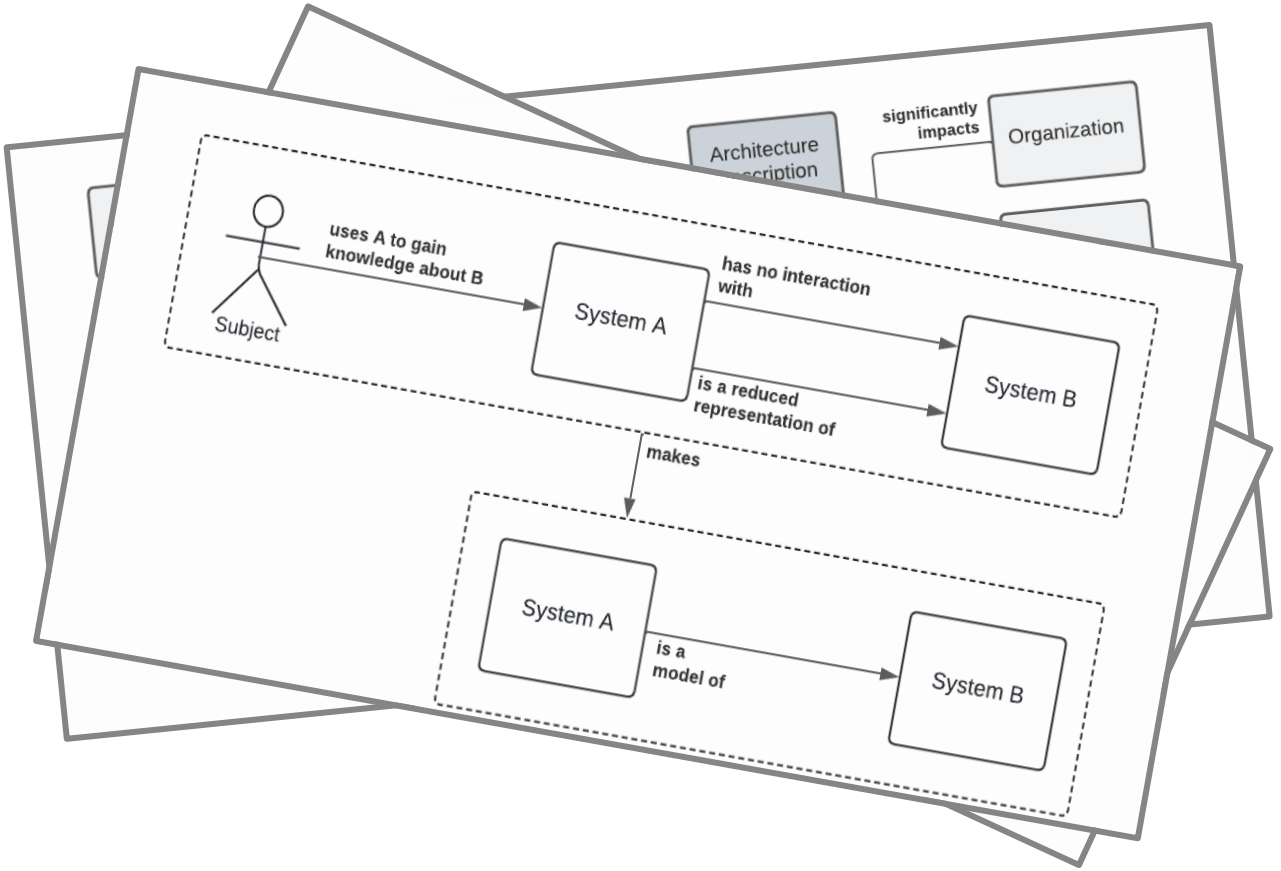
For the **formulation** of **Models**, we use (standardised and widely used) **Modelling Nomenclatures** as a **means** of **manifesting** the **Architecture**.

Incidentally, we have already **looked at** a variety of **concrete Models** up to this point …

… and have always (**inherently**) **used** a **respective Modelling Language** ...

However, **without introducing the Nomenclature beforehand**.

This shows that we have a certain **Modelling Language intuition**.



Architecture View Model

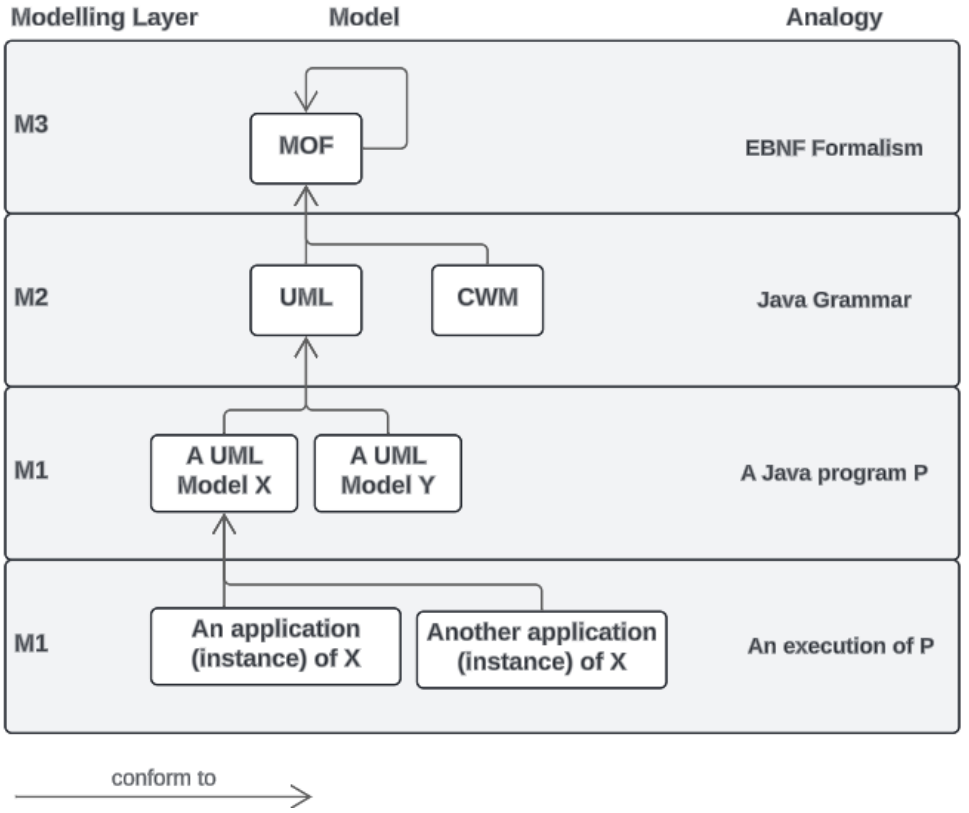
Modelling and Models – Metamodel

A **Metamodel** is a **Model** that **defines** the rules and **structures according to which other Models** can be **built** and **understood**. In short a **Metamodel** is a **Model of a Model**.

The Object Management Group's ([OMG 2010]) **Meta Object Facility (MOF)** nicely **illustrates** what **Metamodels** are.

At the same time, the **MOF** shows that this **concept** is **very relative**. **UML** is both a **metamodel and a model**.

Metamodels define **essential concepts** and **relationships** between them to **enable** the creation of **models based** on the **metamodel language**.



Architecture View Model
Modelling and Models – Contributions

Models have a **huge spectrum** of very **different contributions** that they make. Basically, Architecture cannot make any contributions at all without the aid of Models.

For example, **Models** support **Planning** by revealing **gaps**, **redundancies** or **inadequacies.**

They **help Organizations** to ***do the right things***.

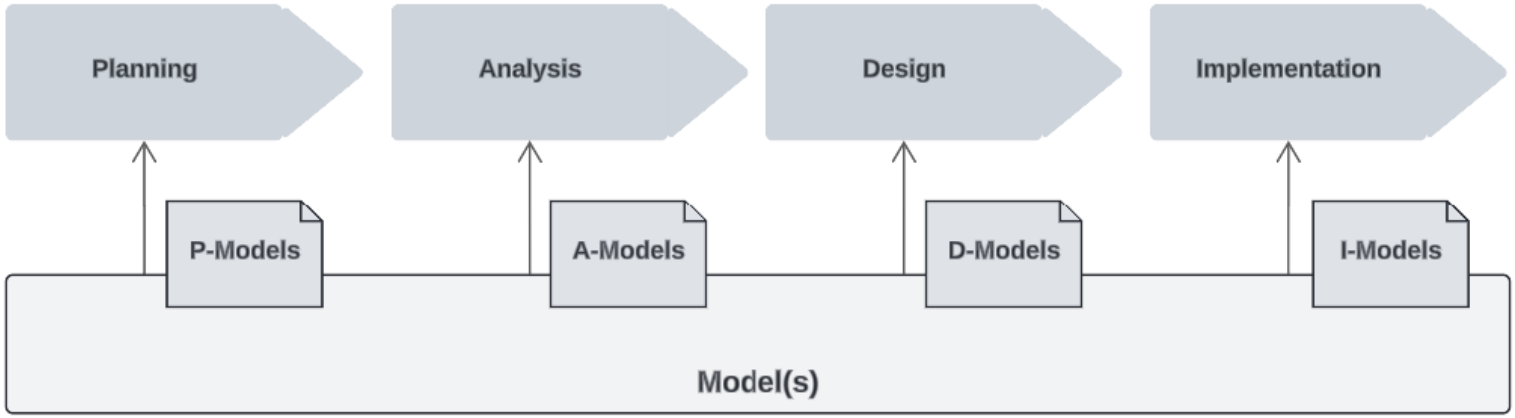
Models support **Analysis by enabling reflection** and **simulation**.
Models are a **basis** for the **cost-effective** implementation of **what-if scenarios** and **impact analyses**.

Models support **Design** processes.

A **Model** is used to **prescribe how something should be done** correctly.

Models are **related** to each other.
For example, a Model at **architecture level** can be further **refined** by a **corresponding Model at design level**.

The **combination** of **Models** in **descending degrees of abstraction** directly **serves** the **Implementation**.

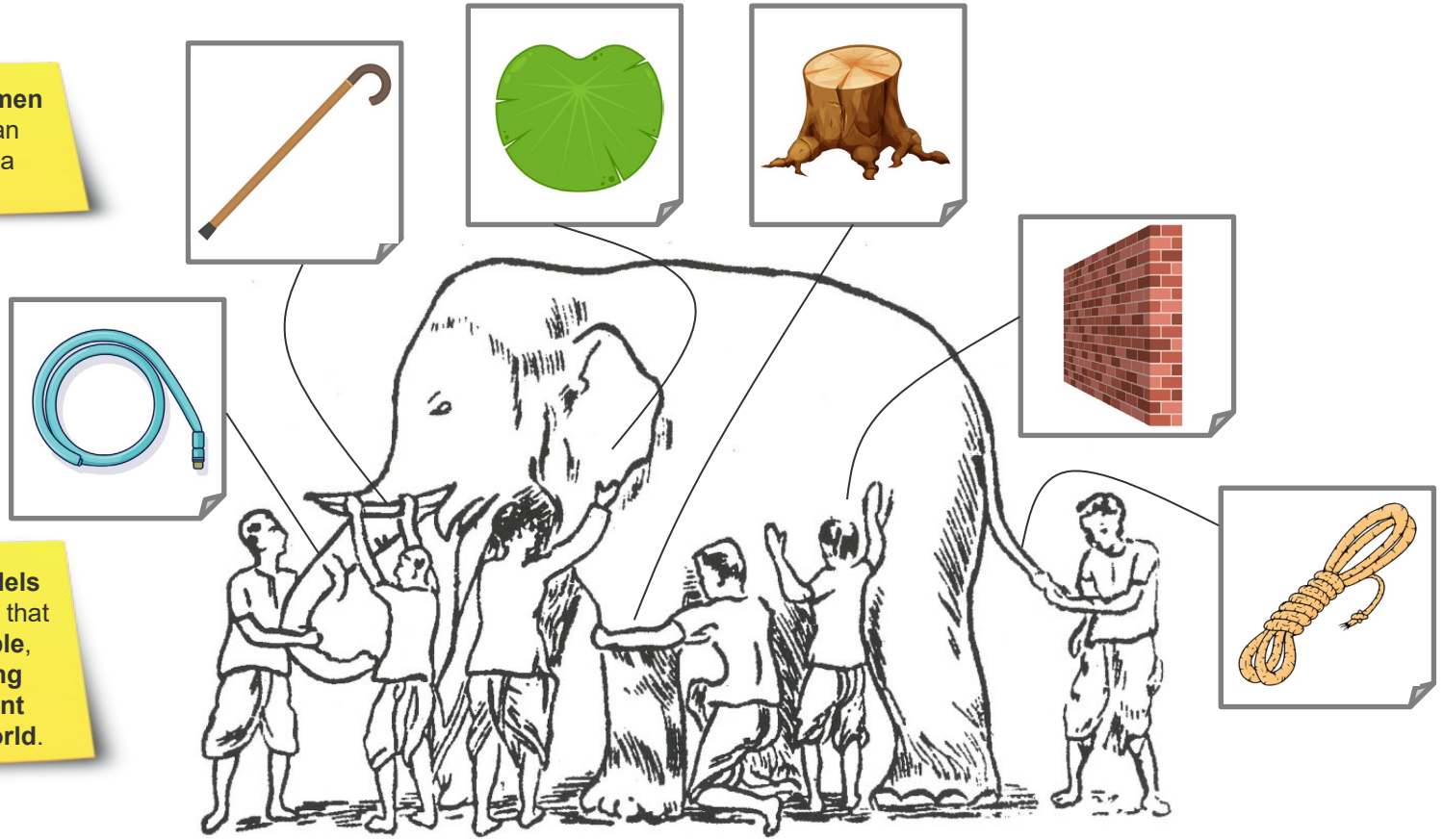


Architecture View Model

Modelling and Models – Misconceptions

What **applies** to **many other things** on planet earth, also **applies to Models**: When **used appropriately**, they are a **blessing**, but **otherwise** sometimes even a **curse**.

As in the **story of the 6 blind men** and the **elephant**, **Models** can completely **underrepresent** a **reality**.



It is **not uncommon** that **Models** are so **vague** and **ambiguous** that they are **completely unusable**, while **at the same time being misunderstood** as a **relevant representation** of the **real world**.

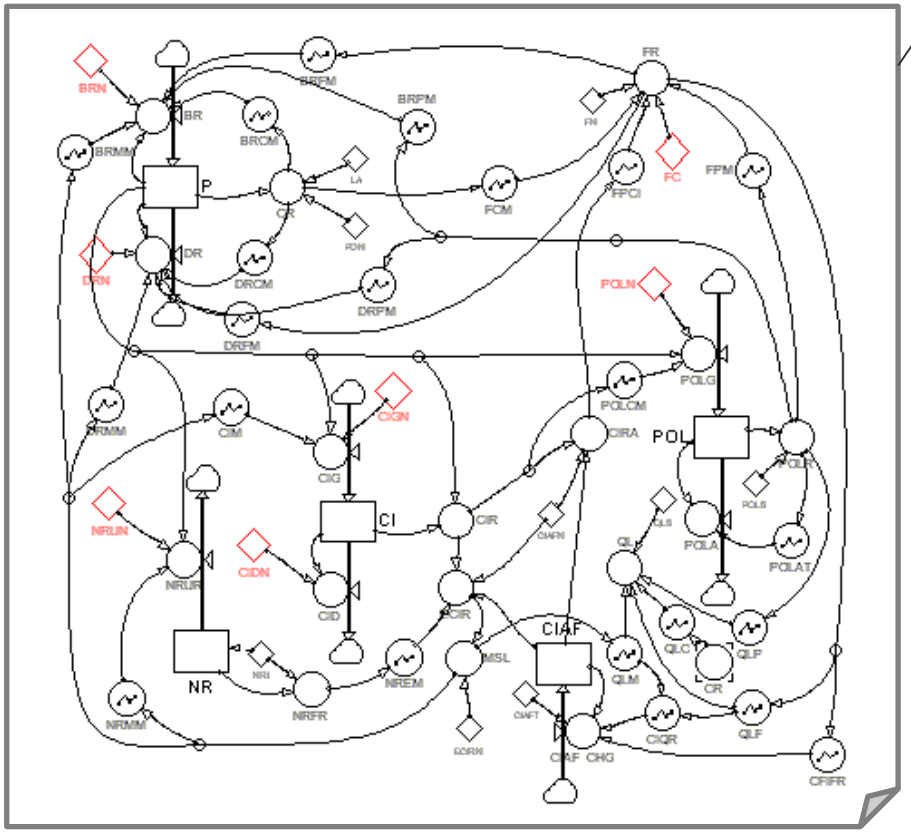
Architecture View Model

Modelling and Models – Misconceptions

What **applies** to **many other things** on planet earth, also **applies to Models**: When **used appropriately**, they are a **blessing**, but **otherwise** sometimes even a **curse**.

At the same time, there are **Models** that **over-represent** a **reality** to such an extent that **they no longer fulfil** their actual **purpose** (reduction of reality to better capture it).

Basically, **all Models are wrong** – just **some** are **more useful than others**.



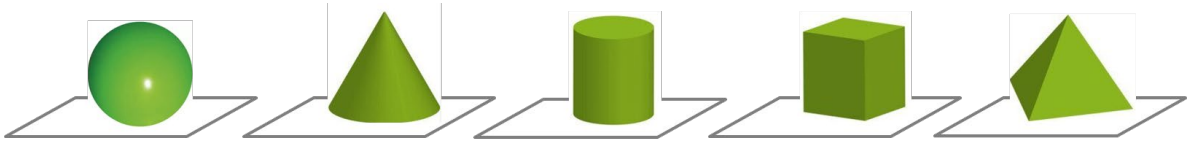
Architecture View Model
View Model

View Models are **essential tools** for **Architects** in general. We will examine selected Architecture View Models, but here we look at a few very **fundamental concepts all View Models share**.

View Models are a **means** of **organising** an **orderly view** of a **complex landscape**.

Landscapes consist of a **broad variety** of different **components** and their **inter-relationships**.

Individual Views of a **View Model** typically **consider specific types of components** (e.g. quadrants) or **component relationships**.

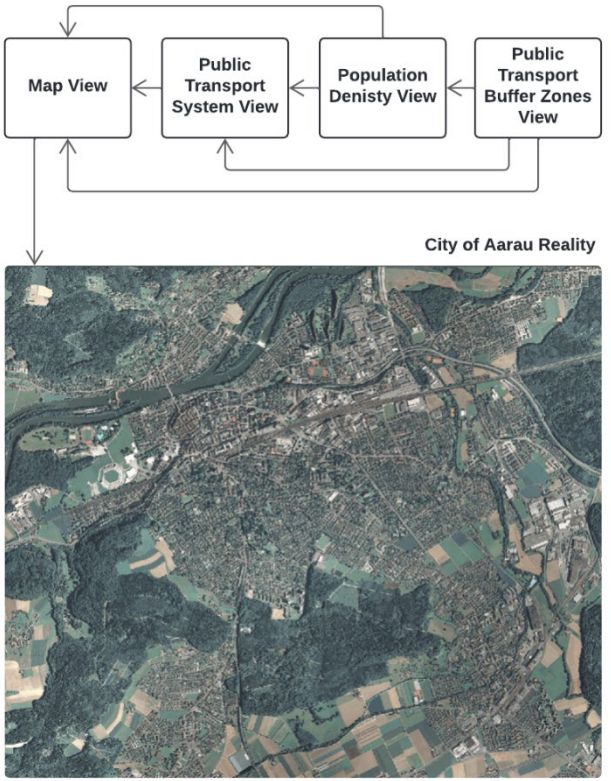


By **explicating** a **systematics** regarding the **usage of Views**, **View Models** allow their users to **consciously select Views** in to gain meaningful **insights into landscapes**.

Architecture View Model
View Model

View Models are **essential tools** for **Architects** in general. We will examine selected Architecture View Models, but here we look at a few very **fundamental concepts all View Models share**.

Let's look at a **small example** of a **View Model** (in the **GIS domain**) consisting of **4 Views + 1 Reality**.



Map View

Architecture View Model
View Model

View Models are **essential tools** for **Architects** in general. We will examine selected Architecture View Models, but here we look at a few very **fundamental concepts all View Models share**.

First, let's take a look at the **Reality of Aarau** – an **aerial photo** of the city of Aarau.

What we are looking at here is **not yet a view** – it is **reality**

Well, we can **criticise** the **above point right away**, since it is a **photograph** – i.e., **a model of reality**.

Map View

Architecture View Model
View Model

View Models are **essential tools** for **Architects** in general. We will examine selected Architecture View Models, but here we look at a few very **fundamental concepts all View Models share**.

We use our good old familiar **Maps** as an **analogy** and look at a **geographical section of Switzerland** (i.e. Aarau) **as the System**.

The **Map reduces** the wide **variety of aspects** that **make up** the **real city of Aarau**.

What **remains recognisable** (i.e., remains) on this map are **urban structures** such as **houses** and **blocks**, **streets**, **railways** and other landscape features such as the **River Aare**.

Public Transport System View

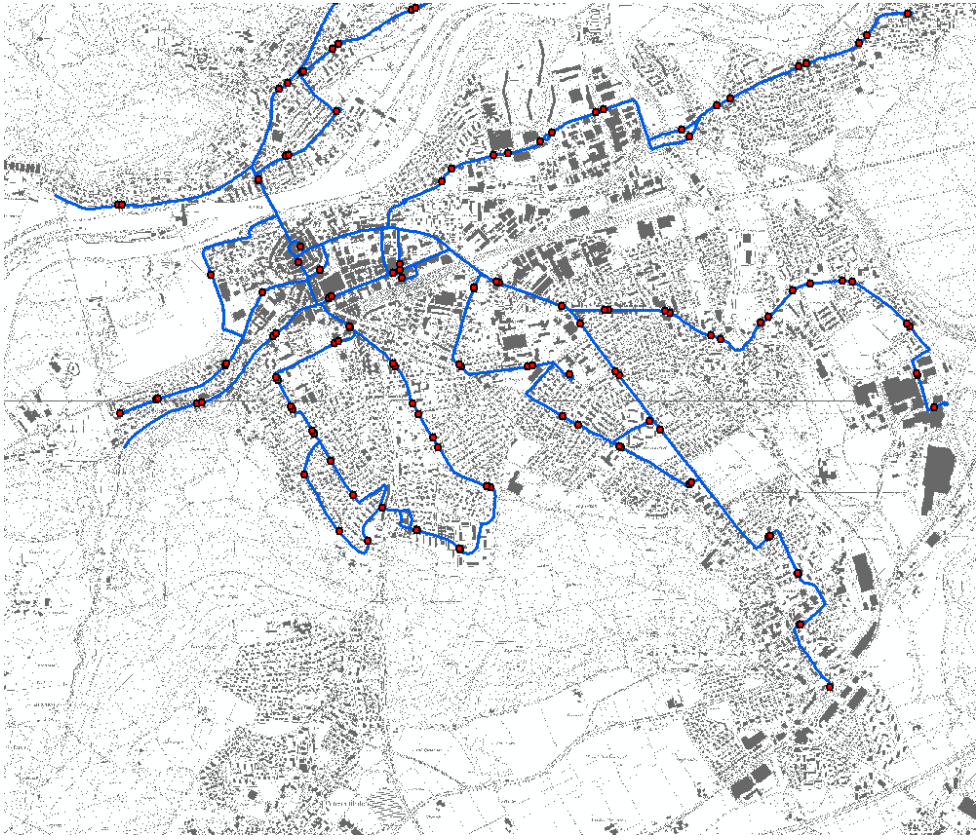
Architecture View Model
View Model

View Models are **essential tools** for **Architects** in general. We will examine selected Architecture View Models, but here we look at a few very **fundamental concepts all View Models share**.

The **diagrams** we have on the right have been **generated by a GIS** (i.e., Geo Information System).

With the help of this system, we have the **local public transport routes** (i.e., tram and rail transport) displayed as **blue lines** and the respective **stops** as **red dots**.

The **nice thing** about this analogy is that the **Public Transport System view** can be **superimposed directly onto the map of the city of Aarau** – **synchronised via geo-coordinates**.



Population Denisty View

Architecture View Model
View Model

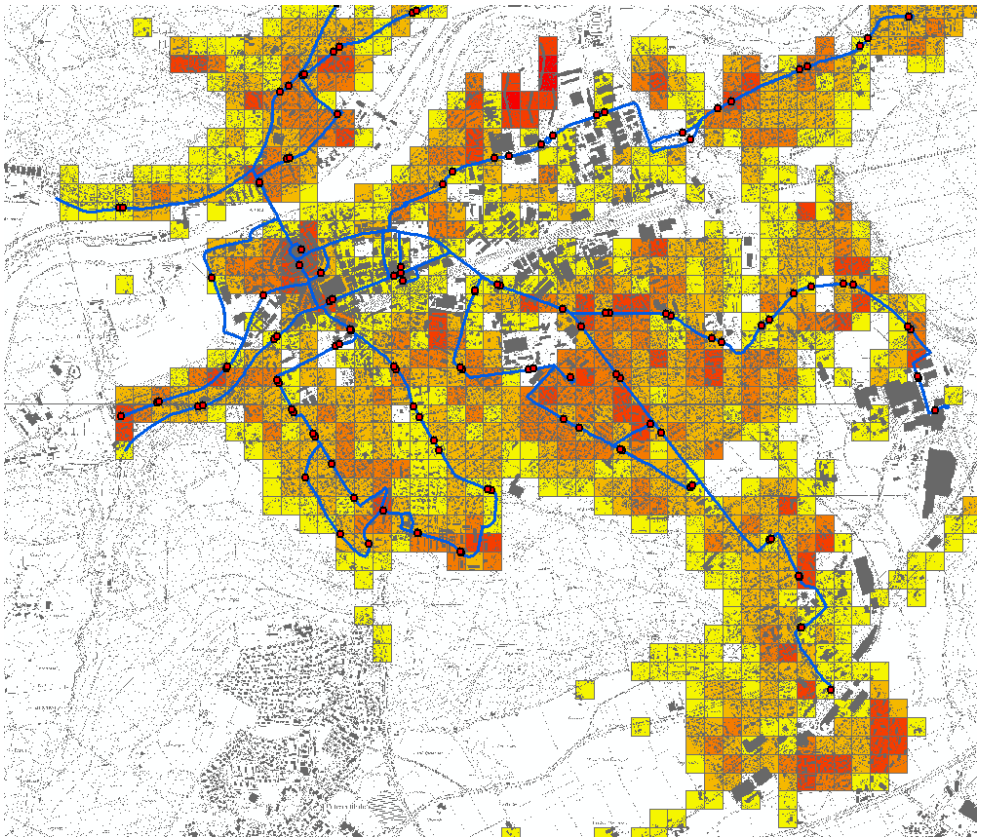
View Models are **essential tools** for **Architects** in general. We will examine selected Architecture View Models, but here we look at a few very **fundamental concepts all View Models share**.

Using the GIS further, we now place a view (**Population Density view**) over the existing views.

The Population Density view **shows the density of the population at a certain grain size** for the entire city of Aarau.

Areas in Aarau marked by **dark red quadrants show a very high population density**, while light yellow quadrants show a very low one, accordingly.

White areas are practically **not populated at all**.



Public Transport Buffer Zones View

Architecture View Model
View Model

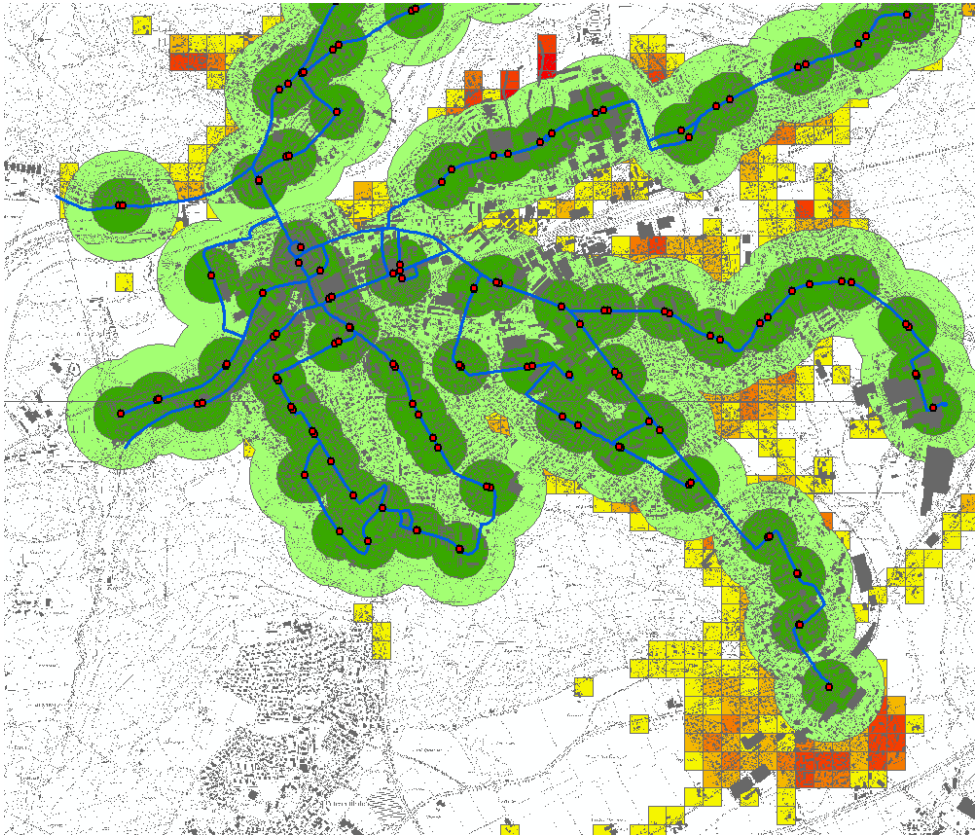
View Models are **essential tools** for **Architects** in general. We will examine selected Architecture View Models, but here we look at a few very **fundamental concepts all View Models share**.

Finally, the GIS gives us **Public Transport Buffer Zones** as a last view.

Like the previous ones, we **place this view on top of the stack of the other views** – like a **transparent foil**.

The new view introduces further information: in which **area** around a tram station is it **very easy to reach the stop** (dark green circle) and in which **area** is the **stop still ok to reach** (light green circle)?

With this analogy we can see that the **power of View Models lies** in the **combination of views** and that view models **favour holistic as opposed to particularistic perspectives**.



Architecture View Model

View Model

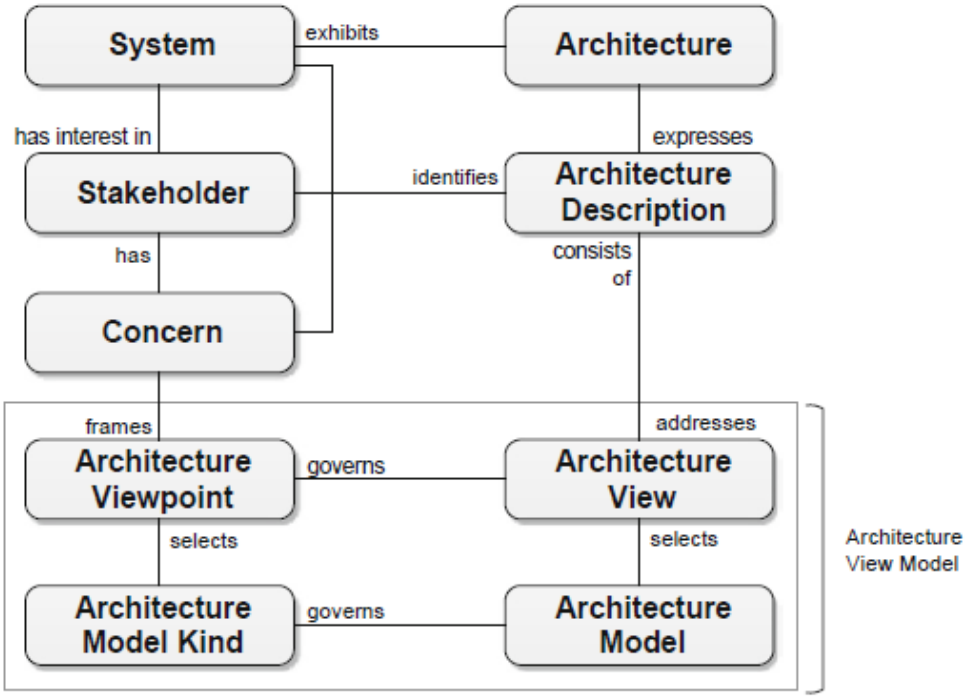
View Models are **essential tools** for **Architects** in general. We will examine selected Architecture View Models, but here we look at a few very **fundamental concepts all View Models sha**

The **IEEE Computer Society** summarises the most **important components** of an **Architecture View Model** in Recommended Practice for Architecture Description of Software-Intensive Systems ([IEEE 2000]).

The **IEEE definition** goes **beyond View Models** and also covers their **broader context**.

The **Architecture manifests** itself in the **form of Architecture Descriptions**.

An **Architecture Description comprises Architecture Views**, with **each View addressing** the **Concerns of** the relevant **Stakeholders**.



Architecture View Model

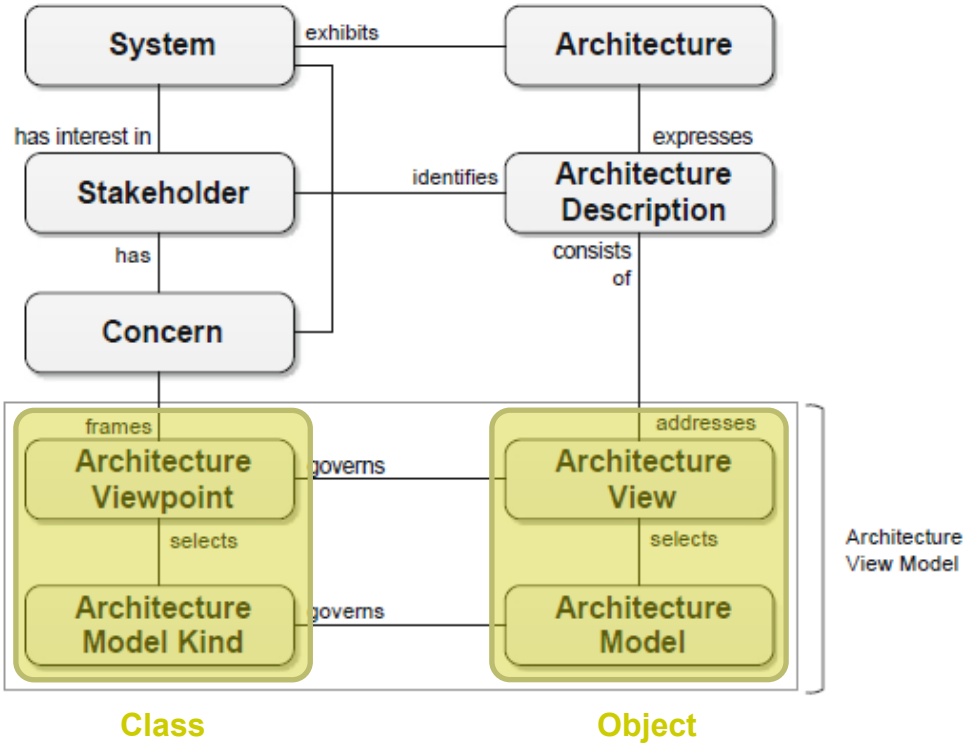
View Model

View Models are **essential tools** for **Architects** in general. We will examine selected Architecture View Models, but here we look at a few very **fundamental concepts all View Models sha**

An **Architecture View consists** of concrete **Architecture Models** that refer to **Architecture Model Types**.

While an **Architecture View** is a **concrete subset** of an **Architecture Description**, an **Architecture Viewpoint** is a set of **conventions for the construction** and **use of Views** derived from it.

A **Viewpoint** comprises **Model Types** and associated **modelling notations** that are **suitable** for the **needs** of the **Viewpoint**.



Architecture Viewpoint relates to **Architecture View**, and **Architecture Model Kind relates** to **Architecture Model** like **Class** relates to **Object** or **Instance**

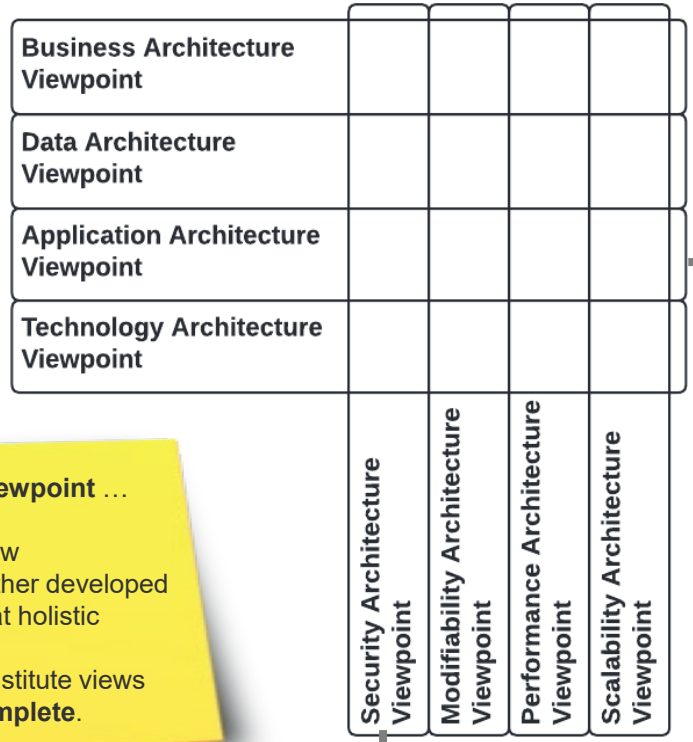
Architecture View Model
View Model

From an Architect's perspective, a **View Model** is a **methodological tool** that helps him to **systematically examine a particular space** (i.e., a solution or landscape of solutions).

A **View Model** can be viewed as **a System of Viewpoints** (including model types) and **relationships between them**.

An **Architecture View Model standardises Architecture Descriptions** by clearly **regulating** the **why**, **what**, **who**, **when** and **how** of the **architecture manifestation**.

A **View Model clarifies** for each **Viewpoint** …
- **whose** interests it serves
- **who** is responsible for creating a view
- **how** views are **instantiated** and further developed
- **how** views are **navigated** to arrive at holistic insights
- **which model types** are used to constitute views
- **when** views can be **considered complete**.



Application Architecture Viewpoint

- **Why**: The viewpoint aims to clarify the required application components and their interaction within a solution.
- **What**: This viewpoint covers the functional and technical aspects of the application architecture, including application components, their interactions, interfaces and relations to components in other viewpoints.
- **Who**: Mainly used by application architects to ensure that the application architecture meets functional requirements as well as enterprise standards.
- **When**: Created in the early stages of solution elaboration and updated throughout the lifecycle of a solution as major changes occur.
- **How**: Models that represent application components, their relationships, interfaces and components in other viewpoints to which application components relate. Models cover dynamic (collaboration) as well as static (whole-part, generalization-specialization) aspects of component relationships

Architecture View Model
View Model

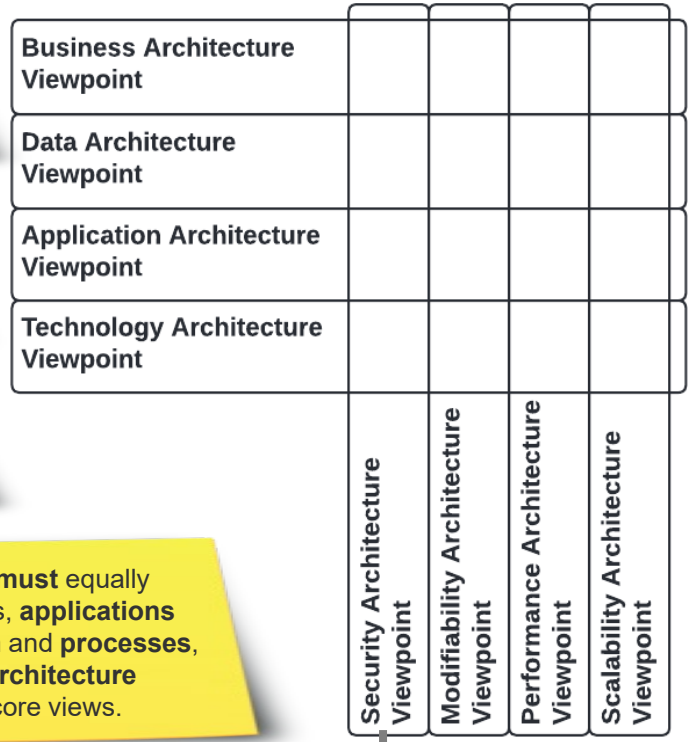
From an Architect's perspective, a **View Model** is a **methodological tool** that helps him to **systematically examine a particular space** (i.e., a solution or landscape of solutions).

In addition to **a set of core Viewpoints**, many **View Models include complementary** (also: cross-cutting) **Viewpoints**.

Although there are many **relationships between Core Viewpoints** of a View Model, **Cross-cutting Viewpoints** go even **a step further**.

For example, seriously taken **security requirements cannot** be **implemented in isolation** at the **Technology Platform** level.

Instead, **security measures must** equally **consider technology** platforms, **applications** and **data** as well as **organisation** and **processes**, which is why the **Security Architecture Viewpoint** cuts across all core views.



Security Architecture Viewpoint

- **Why**: To ensure a consistent and end-to-end security strategy that covers all levels of the architecture, from business processes to technological implementation.
- **What**: This viewpoint integrates security requirements and measures into the entire architecture. It covers topics such as authentication, authorisation, data protection, integrity, availability and compliance across all architectural levels.
- **Who**: Security architects work closely with business analysts, application developers, data architects and technology experts.
- **When**: Begins in the planning stage and is continually evaluated and adjusted throughout the lifecycle of the system, especially when new technologies are introduced or business processes are changed.
- **How**: By developing security guidelines and standards, creating security models and architecture diagrams that show all aspects of security in the different architectural layers.

Architecture View Model
Viewpoint, View, and View Model

A **view model** is a systematics composed of **viewpoints**, **types of models**, and relationships between them.

A **viewpoint** is a definition of the perspective from which a view is taken. It is a specification for constructing and using a view.

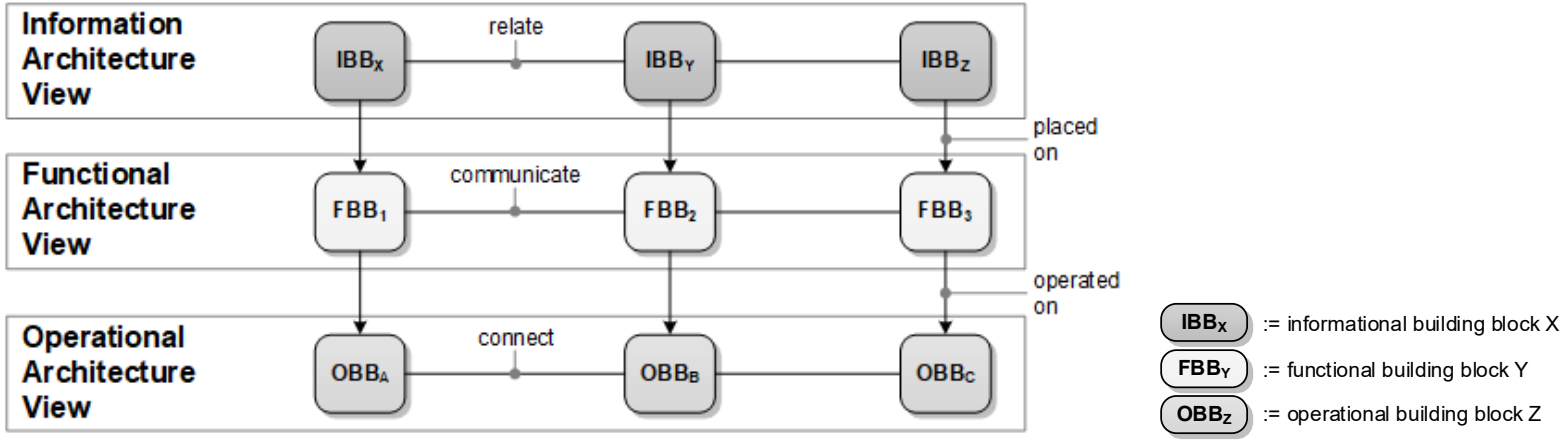
A **view** is what you see, while a viewpoint specifies from where you look. Viewpoints are templates for the concrete views that are instantiated from them.

A view model clarifies for each viewpoint whose interests it serves, who is responsible for creating a view, how views are instantiated and evolved, how views are navigated to arrive at holistic insights, what model types are used to constitute views, or when views can be considered complete.

Architecture View Model
View Model

View models **enable traceability between their viewpoints**—that is, between concrete architecture views and models. By navigating, associating, or combining views, valuable insights emerge, and an architecture description gains plasticity.

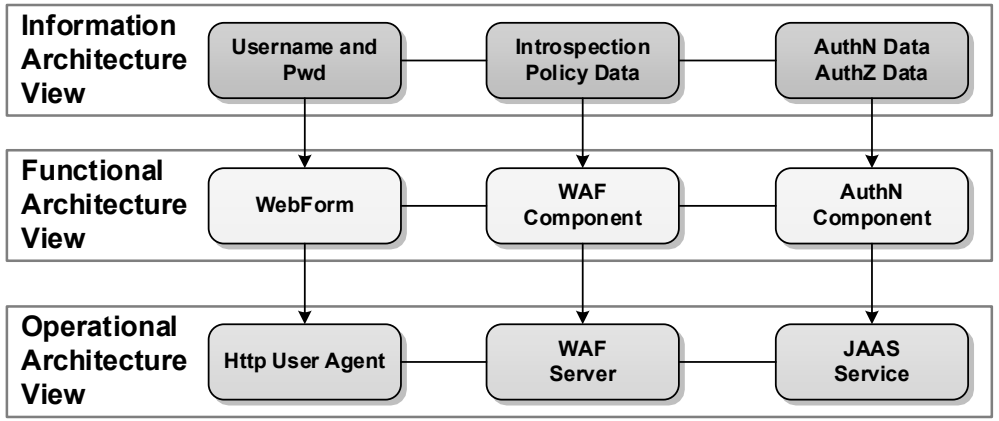
Solution architecture-related view models typically distinguish between three elementary views and differentiate between functional, informational and operational building blocks. Furthermore, such view models clarify elementary relationships that exist between their views.



Architecture View Model
View Model Example

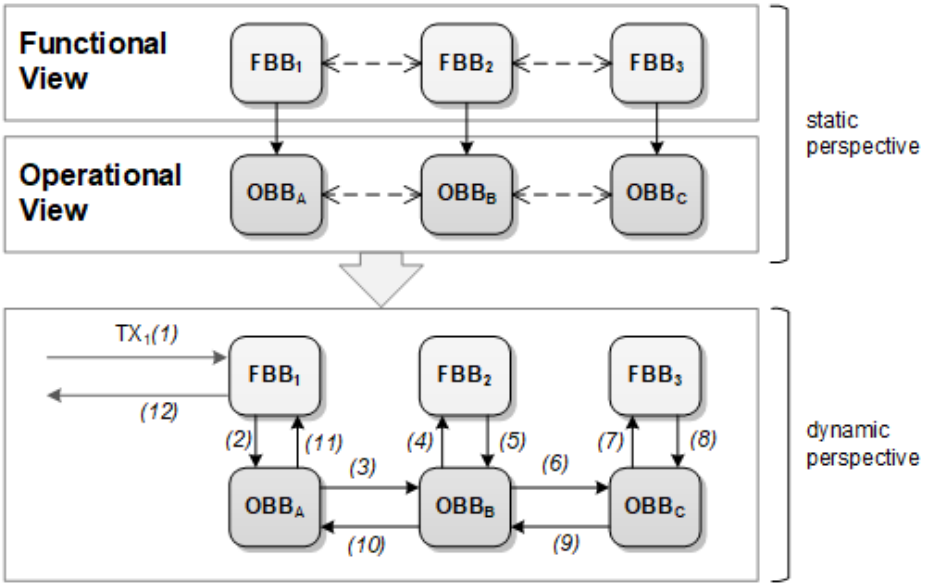
In this **example**, *WebForm*, *WAF Component* and *AuthN Component* are functional components. In a WebForm, *username* and *pwd* are entered. A WAF (i.e., Web Application Firewall) uses *introspection policy data* (e.g., blacklisted characters) to perform input validation of form fields sent via http.

Finally, the AuthN component checks the received AuthN information (username, pwd) against *AuthN data* to verify the authenticity of the user and then against *AuthZ data* to check whether the requested access (i.e. to the website landing page) can be granted.



Architecture View Model
View Model

For example, understanding how functional building blocks are placed on operational building blocks allows you to simulate both, the horizontal and vertical execution threads of a transaction.

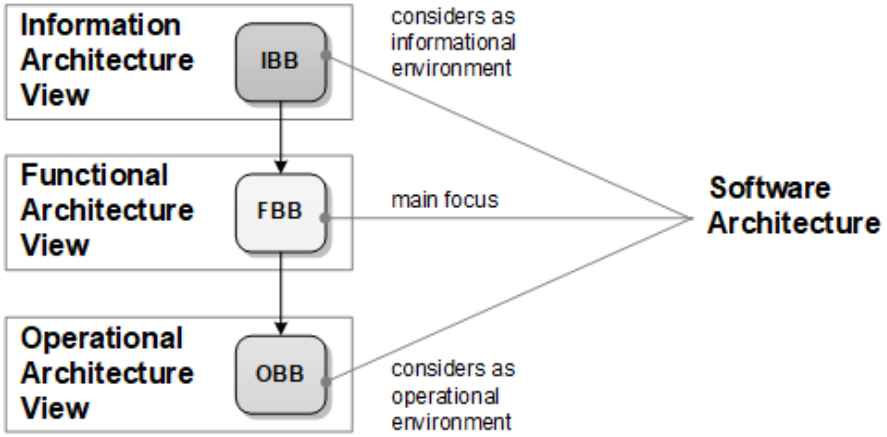


Architecture View Model
View Model

The essential **focus of software architecture** is usually on the **functional building blocks**.

However, the software architect inevitably **considers informational** as well as **operational building blocks** when designing the software architecture.

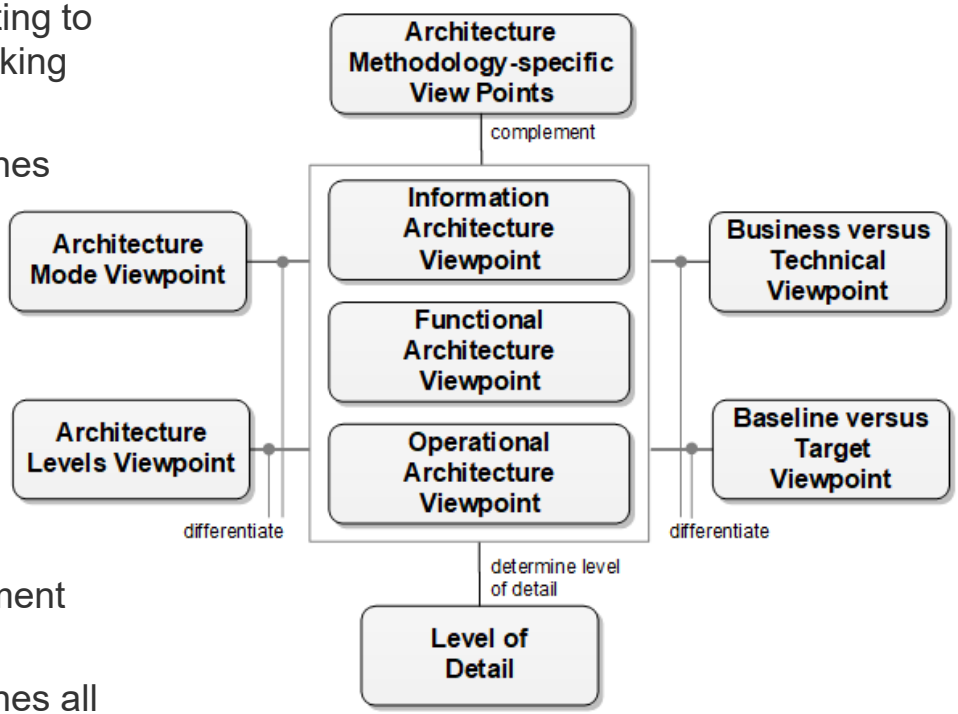
Note on top of that that **operational building blocks are themselves functional building blocks** from the point of view of their manufacturers.



Architecture View Model
Complementary Viewpoints

Complementary viewpoints exist, like …

- **business versus technical viewpoint** distinguishes building blocks directly contributing to business outcomes (business) from those making indirect contributions (technical)
- **baseline versus target viewpoint** distinguishes architecture states (today, 1 year, 3 years, 5 years)
- **architecture mode viewpoint** distinguishes between a design-time, a run-time, and a manage-time architecture perspective
- **architecture methodology specific viewpoints** correspond to, for example, architecture elaboration methods, reference architecture methods, or architecture assessment methods
- **baseline versus target viewpoint** distinguishes all other viewpoints when chronologically different architecture states need to be delineated



Architecture View Model
Complementary Viewpoints

Complementary viewpoints exist, like …

- **level of detail** needs to be calibrated across all viewpoints – it covers degree of abstractness and granularity
  - **solution level** positions landscape assets, like applications, platforms, or basic information entities. Planning horizon at this level is less than one year
  - **domain level** distinguishes structural assets (e.g. domains). Each domain manages its key landscape assets, domain-specific methodological, and reference assets. Planning horizon at the domain level is one to three years
  - **enterprise level** is the level at which you consolidate domains into an enterprise scale perspective. You also manage enterprise-wide methodological and reference assets (e.g. enterprise-wide principles). Planning horizon at enterprise level is strategic, which means three to five years

Architecture View Model

Architecture Frameworks & View Models

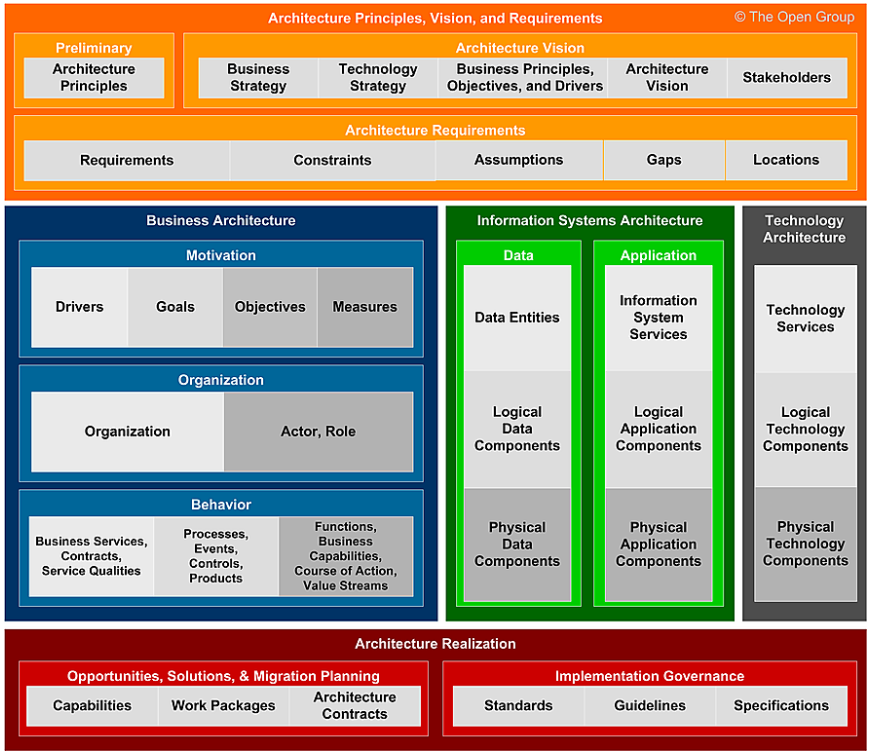
The Open Group Architecture Framework (**TOGAF®**) [TOGAF 2021] proposes an approach to design, plan, implement and govern enterprise architecture by suggesting a common vocabulary, a generic information model, architecture artifacts, and tooling.

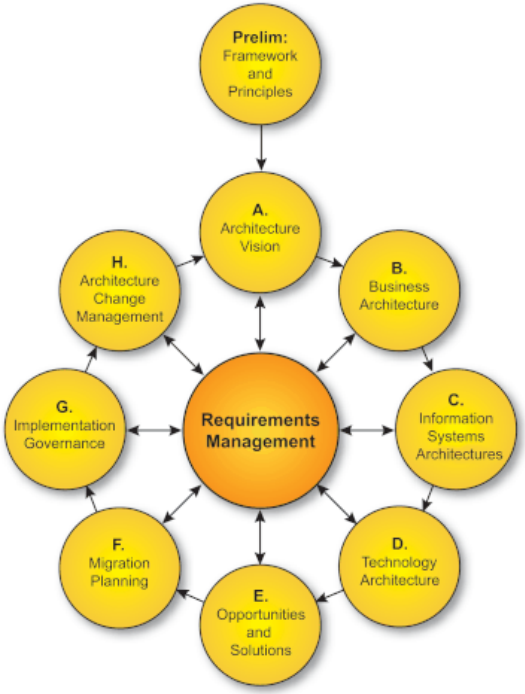
TOGAF, developed by the Open Group, **offers** a **detailed method**, the Architecture Development Method (ADM), for **planning**, **developing** and **maintaining enterprise architectures**.

TOGAF emphasises the architecture process.

However, **TOGAF** is an **Architecture Framework** that has **gained widespread acceptance today**.

It is sometimes **seen** as **too general**, as it **covers a wide range** of **tools** and **techniques without giving specific implementation** or **adaption instructions**.





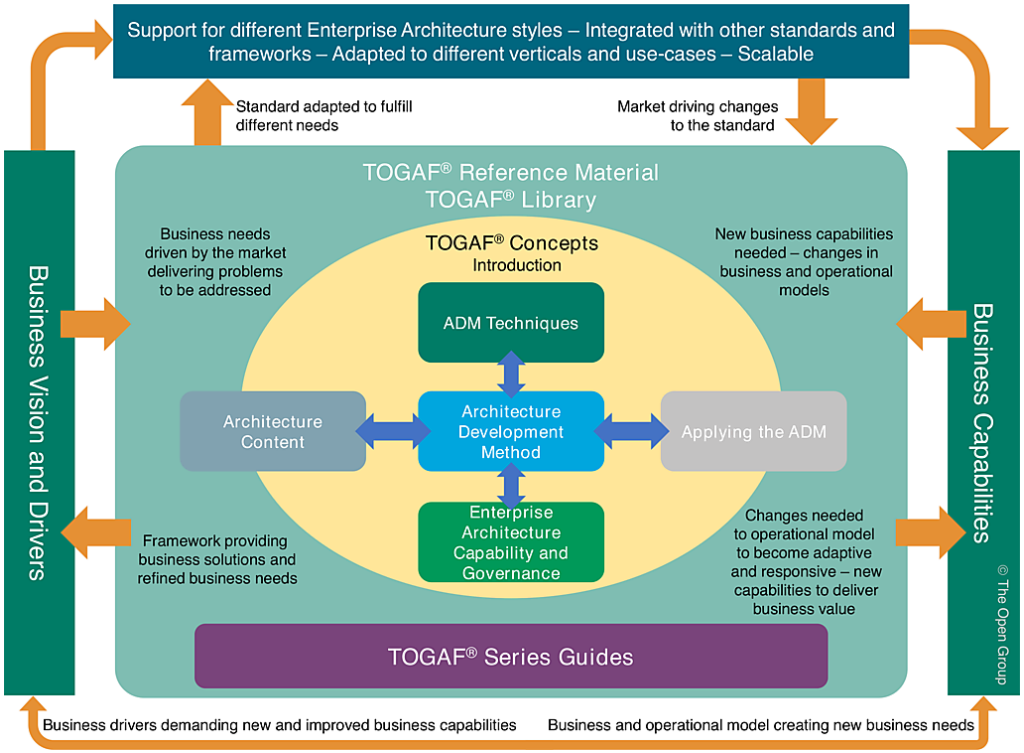
Architecture View Model

Architecture Frameworks & View Models

TOGAF 10 builds on **previous versions** and aims to **support digital transformation**, **promote business agility** and **enable seamless integration** of **modern technologies** such as Cloud and IoT.

Critics continue to **argue** that the **multitude of tools** and **methods provided can be overwhelming** and that **the generic nature** of **TOGAF** can **lead** to **inconsistent implementations** in **organisations**.

However, **TOGAF** is one of the **most widely adopted architecture frameworks** today, is **supported by most tool vendors** and can **be applied at** the **enterprise**, **domain** and **solution architecture levels**.



Architecture View Model

Architecture Frameworks & View Models

ArchiMate® [ArchiMate 2021] is an open and independent modelling standard for domain architectures. It supports the description, analysis, and presentation of architecture within and between designs.

ArchiMate is **more** of a **modelling language than** a **complete Architecture Framework**.

It is **specifically tailored** to the **architecture** of **enterprise systems** and **enables business processes**, **applications** and **technology** layers to be **modelled consistently**.

ArchiMate can be **difficult for beginners** to learn and **introduces** a **meta-model** that is **not always easily applicable** to all **organisations**.

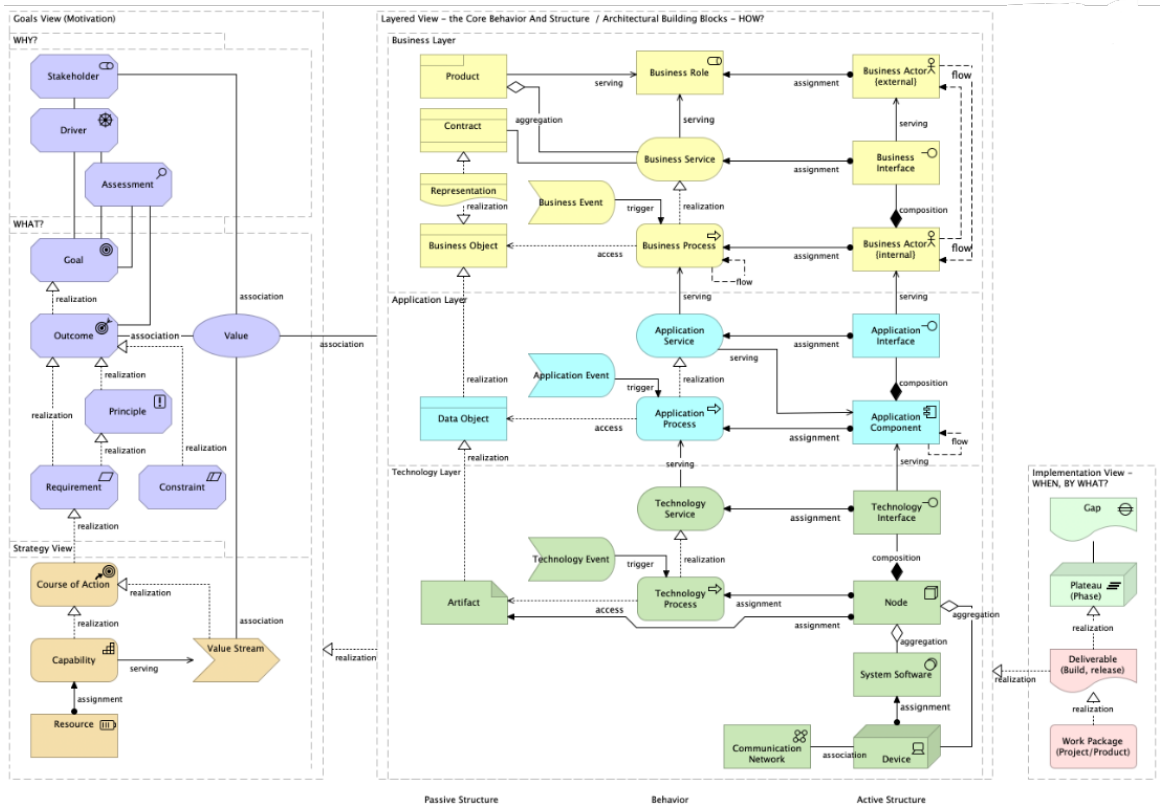
ArchiMate, **like TOGAF**, is an **Open Group framework** and is now **closely integrated with TOGAF**.

Architecture View Model

Architecture Frameworks & View Models

ArchiMate® [ArchiMate 2021] is an open and independent modelling standard for domain architectures. It supports the description, analysis, and presentation of architecture within and between designs.

The **ArchiMate View Model** is **largely self-explanatory** and **structures** the **expression space** that **spans** the **ArchiMate metamodel**.



Architecture View Model

Architecture Frameworks & View Models

ArchiMate® [ArchiMate 2021] is an open and independent modelling standard for domain architectures. It supports the description, analysis, and presentation of architecture within and between designs.

ArchiMate is **more** of a **modelling language than** a **complete Architecture Framework**.

ArchiMate can be **difficult for beginners** to learn and **introduces** a **meta-model** that is **not always easily applicable** to all **organisations**.

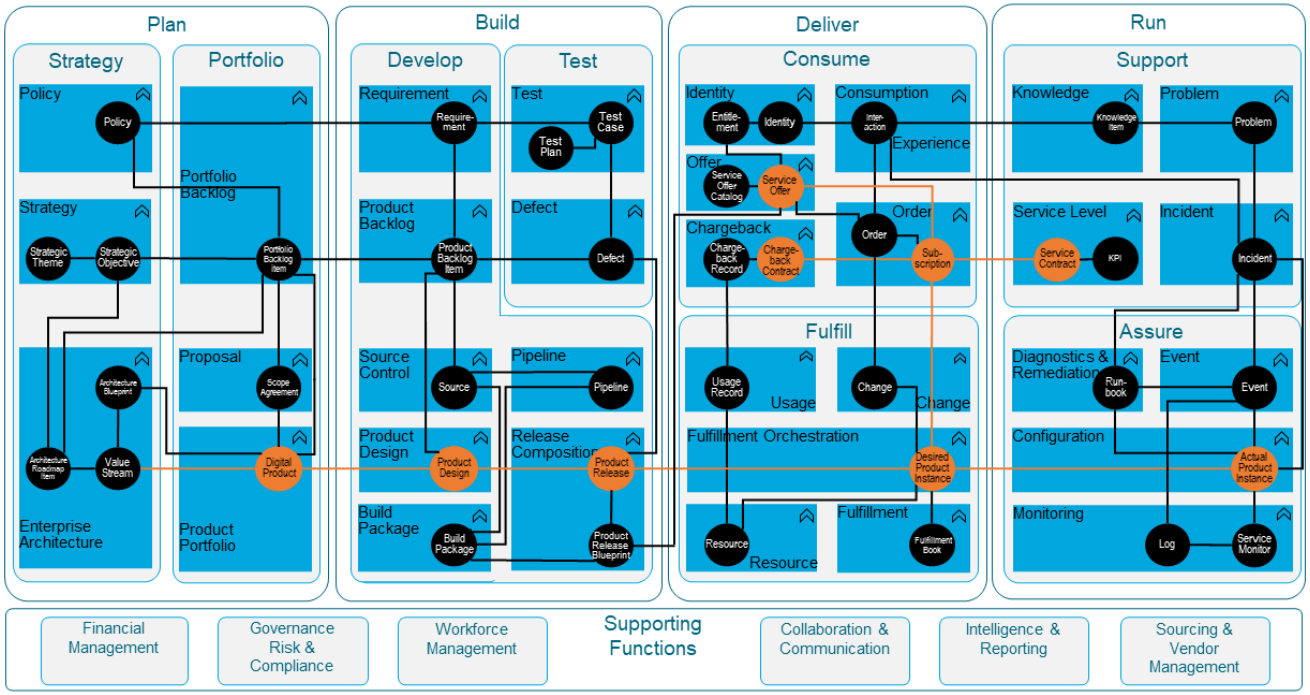
It is **specifically tailored** to the **architecture** of **enterprise systems** and **enables business processes**, **applications** and **technology** layers to be **modelled consistently**.

ArchiMate, **like TOGAF**, is an **Open Group framework** and is now **closely integrated with TOGAF**.

Architecture View Model

Architecture Frameworks & View Models

IT4IT®, developed by the **Open Group**, is very **IT-specific** and **focuses strongly** on **improving IT business processes** by providing an **end-to-end view of IT value streams**.



It is **less** of an **Architecture view model** but **more** of a **blueprint for an IT operating model**.

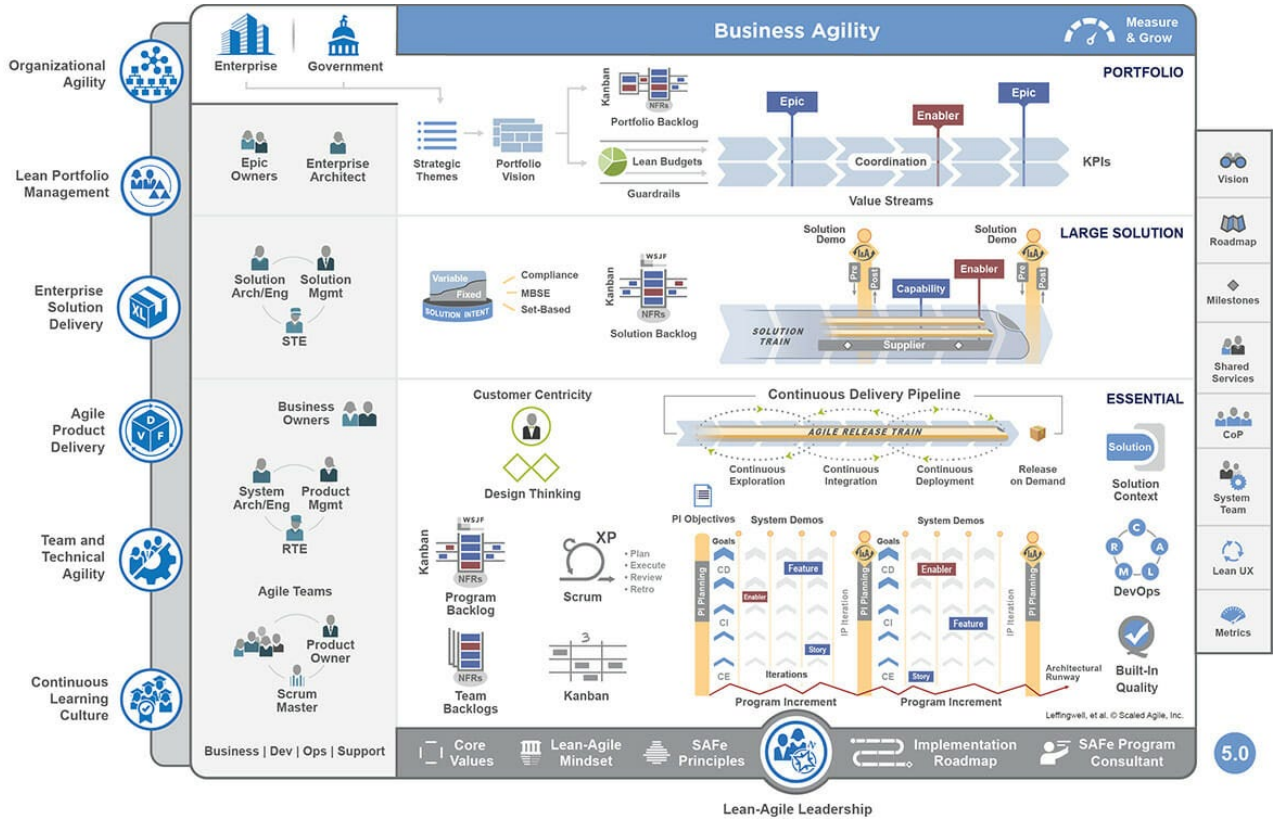
Architecture View Model

Architecture Frameworks & View Models

SAFe combines Lean, **Agile** and **DevOps principles** to provide a **scalable approach** for **operating large organisations**.

SAFe is **less of an EA framework** and **more** of a **blueprint for enterprise operating models**.

SAFe **focuses on** the **coordination** of **multiple teams** and **alignment with business goals**.



Architecture View Model

Architecture Frameworks & View Models

The **Zachman Framework** [Zachman 2021] (the old timer among the models) proposes a structured approach to holistically viewing and defining an enterprise's architecture. The basic scheme distinguishes two dimensions that introduce the intersection of two classifications.

The **Zachman Framework** is the **first formal architecture framework** that offers a **comprehensive approach** to the **organisation** of **architecture descriptions**.

It can **be understood** as a **comprehensive view model without** a **particular focus** on the **architecture process**.

	DATA *What*	FUNCTION *How*	NETWORK *Where*	PEOPLE *Who*	TIME *When*	MOTIVATION *Why*	
SCOPE (CONTEXTUAL) *Planner*	List of Things Important to the Business ENTITY = Class of Business Thing	List of Processes the Business Performs Process = Class of Business Process	List of Locations in which the Business Operates Node = Major Business Location	List of Organizations Important to the Business People = Major Organization Unit	List of Events/Cycles Significant to the Business Time = Major Business Event/Cycle	List of Business Goals/Strategies Ends/Means = Major Business Goal/Strategy	SCOPE (CONTEXTUAL) *Planner*
BUSINESS MODEL (CONCEPTUAL) *Owner*	e.g. Semantic Model Ent = Business Entity Reln = Business Relationship	e.g. Business Process Model Proc. = Business Process I/O = Business Resources	e.g. Business Logistics System Node = Business Location Link = Business Linkage	e.g. Work Flow Model People = Organization Unit Work = Work Product	e.g. Master Schedule Time = Business Event Cycle = Business Cycle	e.g. Business Plan End = Business Objective Means = Business Strategy	BUSINESS MODEL (CONCEPTUAL) *Owner*
SYSTEM MODEL (LOGICAL) *Designer*	e.g. Logical Data Model Ent = Data Entity Reln = Data Relationship	e.g. Application Architecture Proc = Application Function I/O = User Views	e.g. Distributed System Architecture Node = I/S Function (Processor, Storage, etc) Link = Line Characteristics	e.g. Human Interface Architecture People = Role Work = Deliverable	e.g. Processing Structure Time = System Event Cycle = Processing Cycle	e.g., Business Rule Model End = Structural Assertion Means = Action Assertion	SYSTEM MODEL (LOGICAL) *Designer*
TECHNOLOGY MODEL (PHYSICAL) *Builder*	e.g. Physical Data Model Ent = Segment/Table/etc. Reln = Pointer/Key/etc.	e.g. System Design Proc = Computer Function I/O = Data Elements/Sets	e.g. Technology Architecture Node = Hardware/Systems Software Link = Line Specifications	e.g. Presentation Architecture People = User Work = Screen Format	e.g. Control Structure Time = Execute Cycle = Component Cycle	e.g. Rule Design End = Condition Means = Action	TECHNOLOGY MODEL (PHYSICAL) *Builder*
DETAILED REPRESENTATIONS (OUT-OF-CONTEXT) *Sub-Contractor*	e.g. Data Definition Ent = Field Reln = Address	e.g. Program Proc = Language Statement I/O = Control Block	e.g. Network Architecture Node = Address Link = Protocol	e.g. Security Architecture People = Identity Work = Job	e.g. Timing Definition Time = Interrupt Cycle = Machine Cycle	e.g. Rule Specification End = Sub-condition Means = Step	DETAILED REPRESENTATIONS (OUT-OF-CONTEXT) *Sub-Contractor*
FUNCTIONING ENTERPRISE	e.g. DATA	e.g. FUNCTION	e.g. NETWORK	e.g. ORGANIZATION	e.g. SCHEDULE	e.g. STRATEGY	FUNCTIONING ENTERPRISE

© John A. Zachman, Zachman International

It uses a **matrix structure** that **combines** different **systemic perspectives** (e.g., data, function, network) and **different levels of abstraction** or **detail** (e.g., business, system, technology) to **systematize architecture representations**.

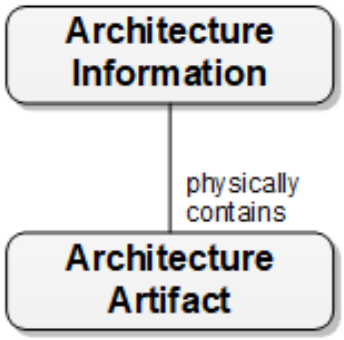
Today, it is **considered outdated**, is **rigid** and **complex in its application** and **offers no support** for **modern agile methods**, for example.

Architecture Description

Architecture artifacts and architecture information

Architecture manifests in architecture descriptions (aka: architecture artifacts, architecture work products).

Artifacts can be more or less structured and formalized. They are **means by which architects process architecture information in a tangible manner**.



Architecture Description
Architecture artifacts and architecture information

Physically **tangible architecture information** can be accessed, sent, and exchanged between cooperating parties in a controlled and secure manner.

Artifacts protect their content from unauthorized access. They can be signed, versioned, archived, and physically referenced from other artifacts.

Artifacts are agreed upon as deliverables, whose delivery can be efficiently quantified and verified.

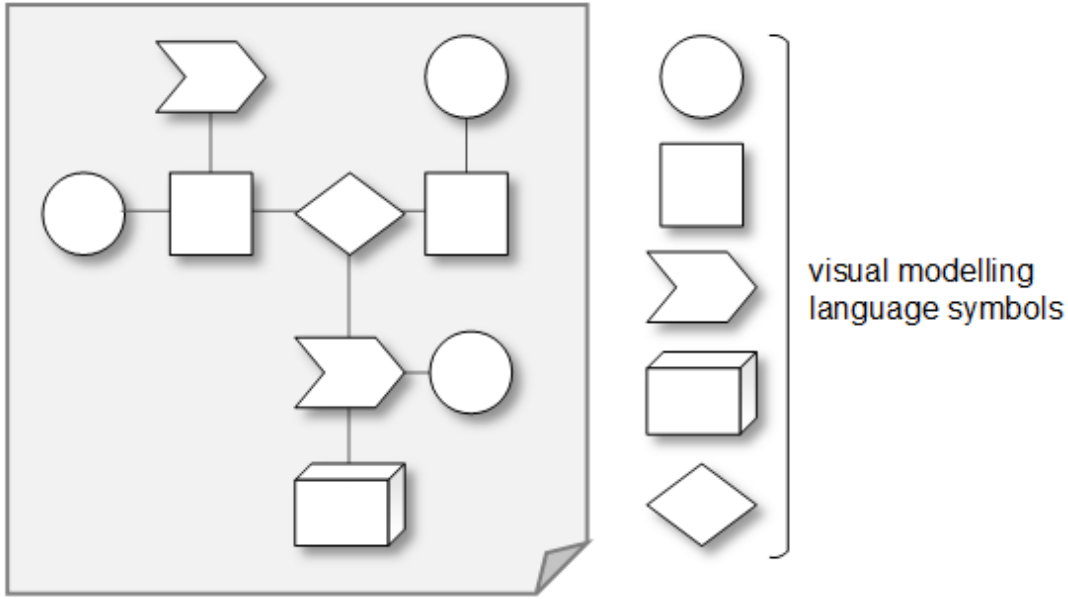
Artifacts can represent final information, or they can be dynamically generated based on an architecture repository.

Architecture information – structured, unstructured, or semi-structured – is compiled into artifacts and can take **various forms**, **including text**, **diagrams**, **lists**, **or matrices.** For example, statements formulated in plain English take the text form. However, text can also take the form of a formal language, such as a pseudocode fragment, which expresses the algorithmic structure of an asset's capability.

Architecture Description

Architecture artifacts and architecture information

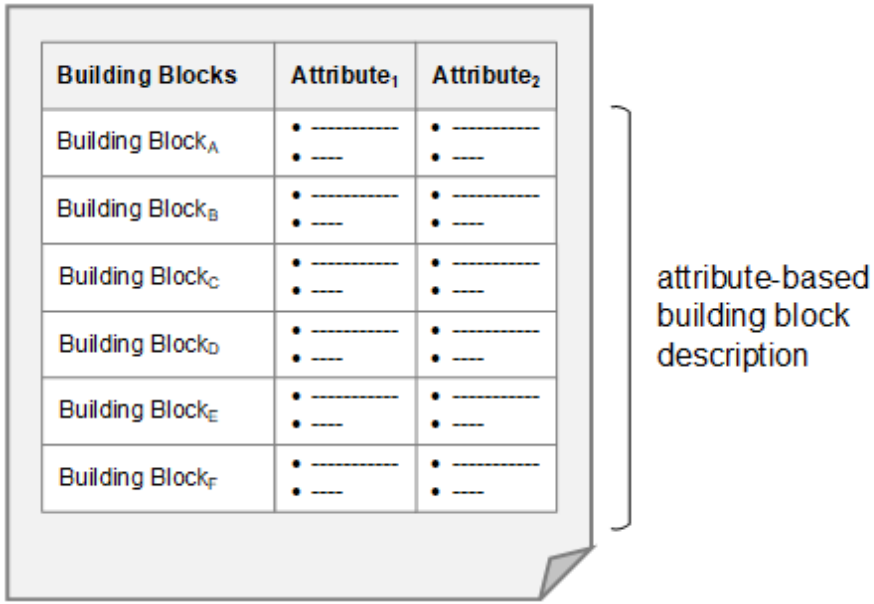
Illustrations and diagrams exist in a variety of visual nomenclatures and modelling languages, each with its own standardized visual symbols, such as the Unified Modelling Language (UML).



Architecture Description

Architecture artifacts and architecture information

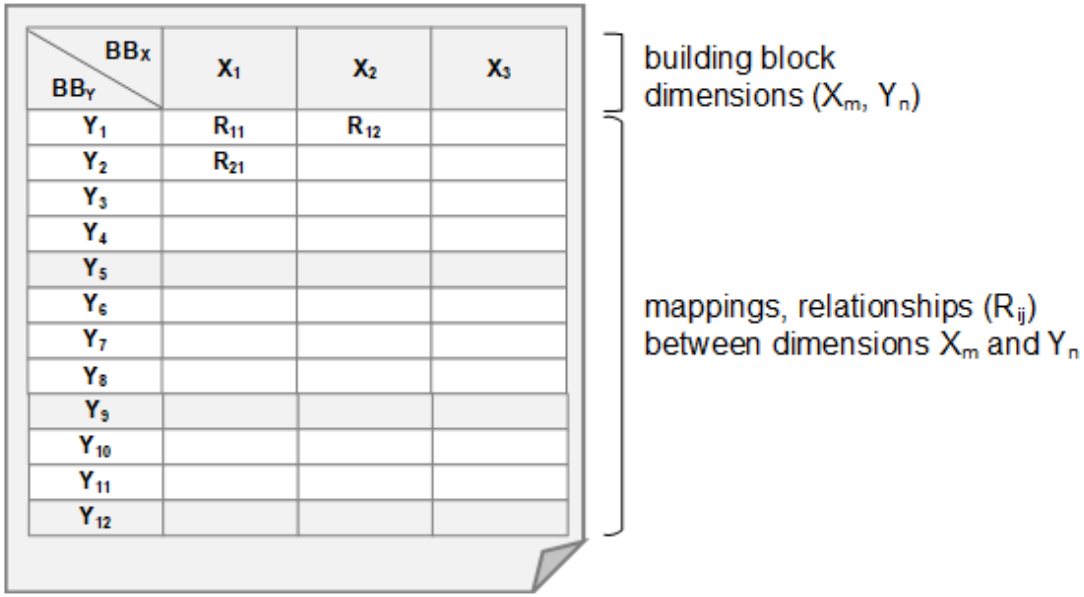
Lists are another common form of representing architecture information. A pattern catalog, a report on all architecture assets in the purchasing domain, or a portfolio viewpoint in a software architecture method are examples of artifacts that take list form



Architecture Description

Architecture artifacts and architecture information

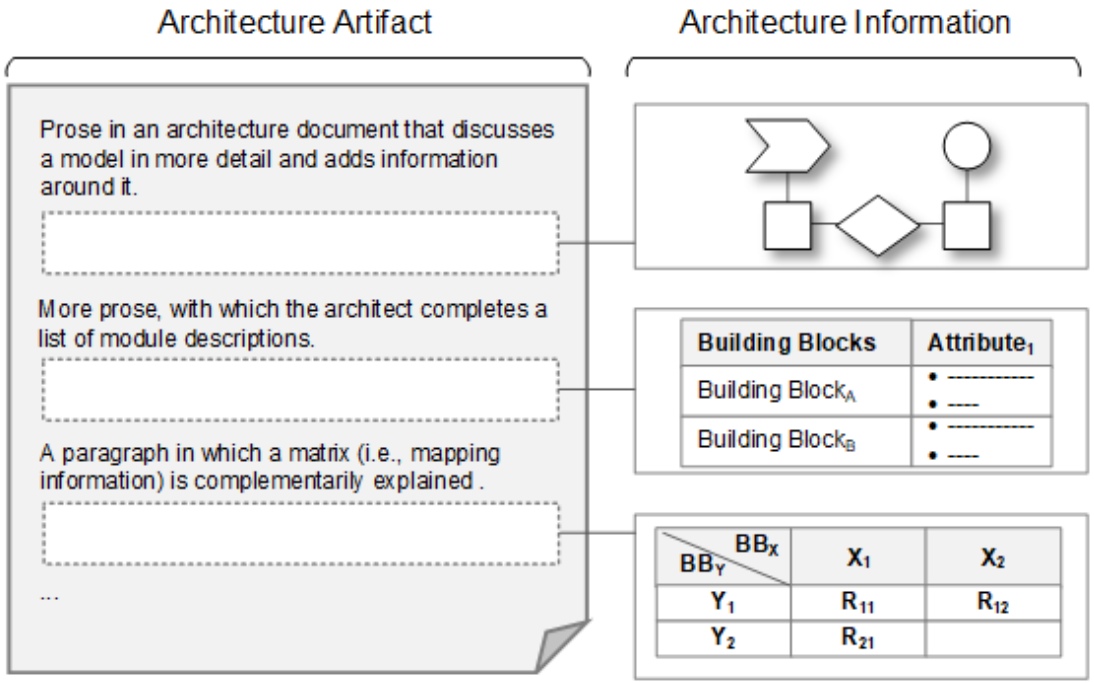
Matrices is a universally used form that maps two architecturally relevant information dimensions, unidirectionally or bidirectionally, to each other. For example, which services are supported by a considered application.



Architecture Description

Architecture artifacts and architecture information

Architecture **information from many sources and in different forms is finally combined into an overarching architecture artifact**.



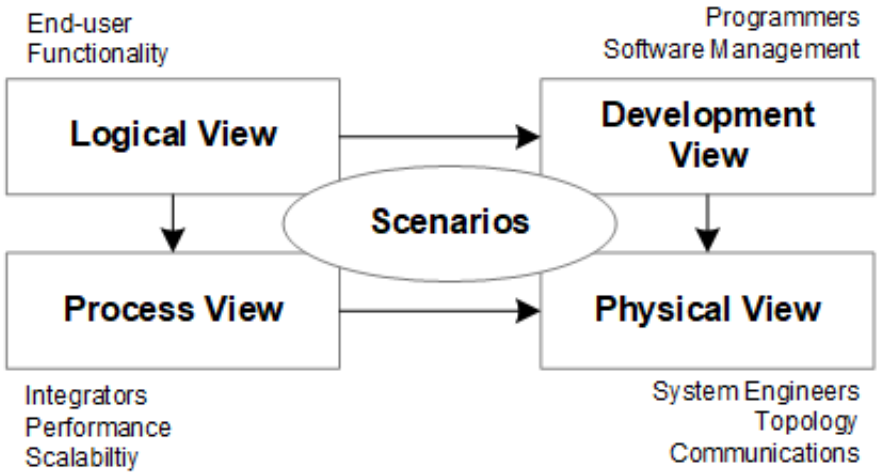
Lecture Agenda

- Architecture View Model
- Kruchten 4+1 View Model
- Development View to Logical View

Architecture – Kruchten 4+1 View Model
Overview

The view of **software architecture manifestation** as a total of different views goes back to the work of Philippe Kruchten [Kruchten 1995].

The **four core views are complemented by one** (hence +1) specification view (scenarios). The arrows between two views mean that the view at the beginning of the arrow influences the view at the end of the arrow.



Architecture – Kruchten 4+1 View Model
Overview

The **logical view** represents cooperation relationships of components—that is, a representation of how functional components (e.g., objects) cooperate to realize scenarios.

The **development view** describes the static organization of components in their development environment—thus a representation of which design time structures (e.g., classes) factorize the logical view.

The **process view** captures concurrency and synchronization aspects of the design—that is, a representation of how components are grouped into processes and what cooperative relationships exist between processes.

The **physical view** describes mapping(s) of software to hardware, or an underlying operating layer, and corresponding aspects of software distribution—that is, a representation of how software and hardware ultimately form an overarching operational unit.

example

Architecture – Kruchten 4+1 View Model

Example 1

In this example Kruchten's 4 + 1 view model is utilized to abstract from a linked list code snippet in Java.


example

Architecture – Kruchten 4+1 View Model
Example 1

Take the following Linked List implementation as an example. Represent the architecture of this software in both the logical, development, and physical view.

As a use case, consider this scenario: elements “1”, “2”, and “3” are *added* to *List<String>*. Next the *contains* method is called with a parameter “2”. Finally the *plot* method is called.

```
public class Node <T> {
	private T element;
	private Node<T> next;

	public Node (T element, Node<T> next) {
		this.element = element; this.next = next;
	}
	public T getElement() {
		return element;
	}
	public Node<T> getNext() {
		return next;
	}
	public void setNext(Node<T> next) {
		this.next = next;
	}
}
```

Architecture – Kruchten 4+1 View Model
Example 1

Take the following Linked List implementation as an example. Represent the architecture of this software in both the logical, development, and physical view.

```
public class List <T> {
	private Node<T> first = null;
	private Node<T> current = null;

	public void add(T element) {
		Node<T> n = new Node<T> (element, null);

		if(first == null) {
			first = current = n;
		} else {
			current.setNext(n); current = n;
		}
	}
	public boolean contains(T element) {
		Node<T> n = first;
		while (n!=null && !n.getElement().equals(element)) { n = n.getNext(); }
		return n != null;
	}
	public String plot() {
		String result = new String(); Node<T> n = first;
		while(n!=null) { result += n.getElement().toString() + " "; n = n.getNext(); }
		return result;
	}
}
```

Architecture – Kruchten 4+1 View Model
Example 1

Take the following Linked List implementation as an example. Represent the architecture of this software in both the logical, development, and physical view.

```
public class Client {
	public static void main(String[] args) {
		List<String> list = new List<String>();

		list.add("1");
		list.add("2");
		list.add("3");

		System.out.println(list.contains("2"));
		System.out.println(list.plot());
	}
}
```

example

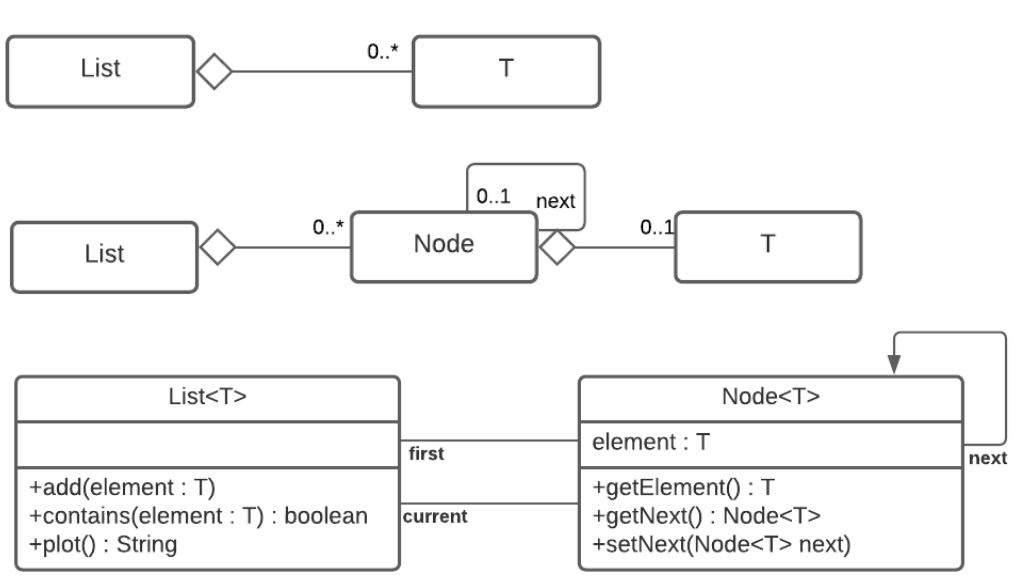
Architecture – Kruchten 4+1 View Model
Example 1

How is this represented in the development view?

example

Architecture – Kruchten 4+1 View Model
Example 1

How is this represented in the development view?



Black-box view

White-box views

example

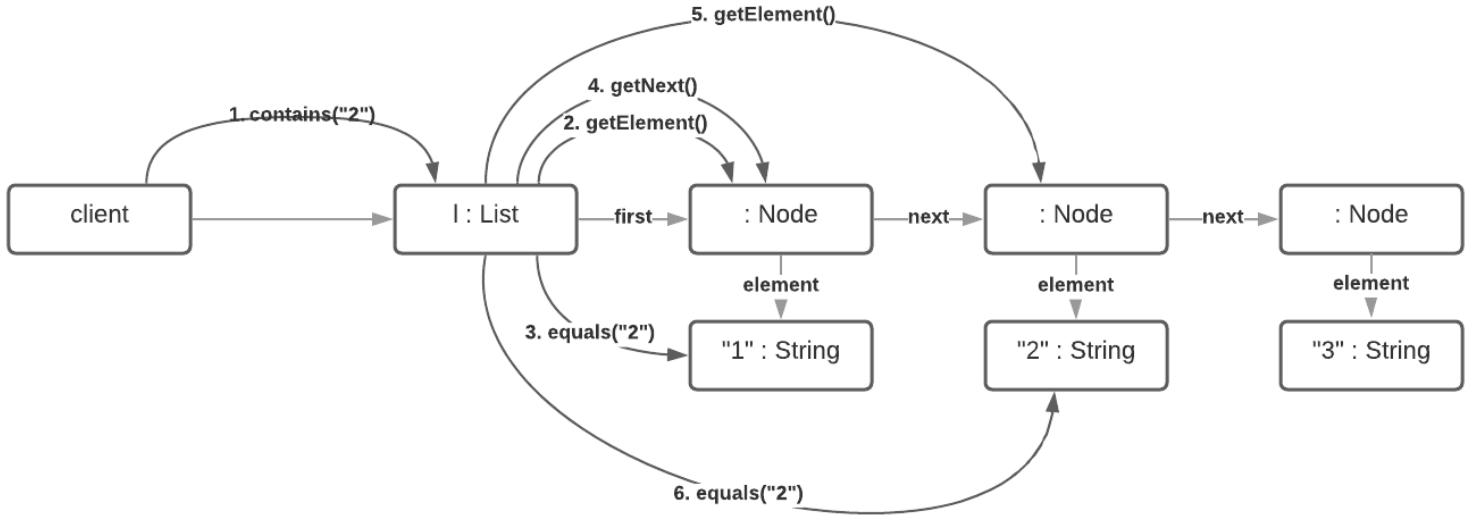
Architecture – Kruchten 4+1 View Model
Example 1

How is this represented in the logical view?

example

Architecture – Kruchten 4+1 View Model
Example 1

How is this represented in the logical view?



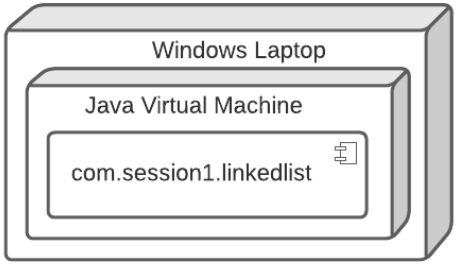
example

Architecture – Kruchten 4+1 View Model
Example 1

How is this represented in the physical view?

Architecture – Kruchten 4+1 View Model
Example 1

How is this represented in the physical view?



example

Architecture – Kruchten 4+1 View Model
Example 2

You are asked to design a data type Stack by transforming the scenario I provide you below into a logical view of Kruchten's view model.

Next, you will develop a suitable development view. Once you are satisfied with your design, you will also code your Stack design in Java.

Please use this stack scenario as input:

1. You add the element "stack" to an empty stack (push).
2. You add the element "first" to the stack (push).
3. You add the element "My" to the stack (push).
4. As long as the stack is not empty (isEmpty) ...
    1. read the topmost stack element (top)
    2. remove the topmost stack element (pop)

example

Architecture – Kruchten 4+1 View Model

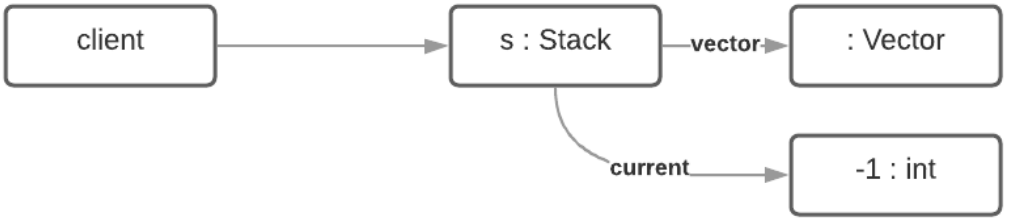
Example 2

How do you transfer the stack scenario into the Logical View of Kruchten's View Model?

Architecture – Kruchten 4+1 View Model
Example 2

How do you transfer the stack scenario into the Logical View of Kruchten's View Model?

Empty Stack state

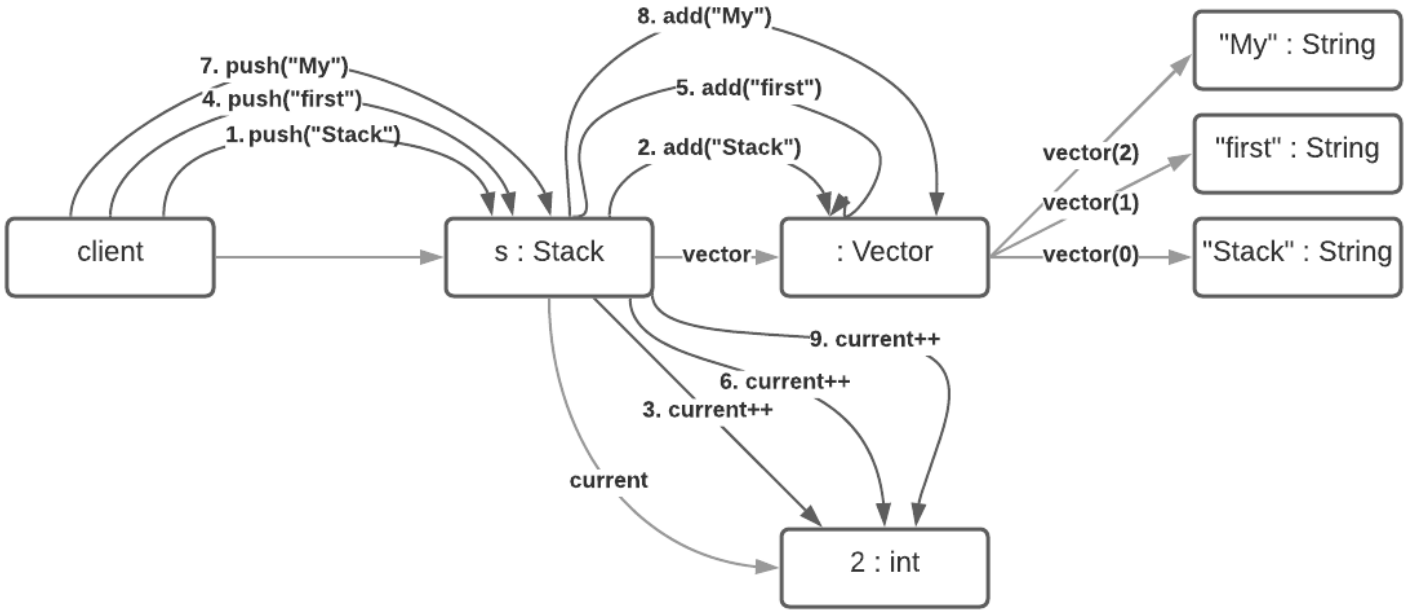


example

Architecture – Kruchten 4+1 View Model
Example 2

How do you transfer the stack scenario into the Logical View of Kruchten's View Model?

Loading the Stack with elements

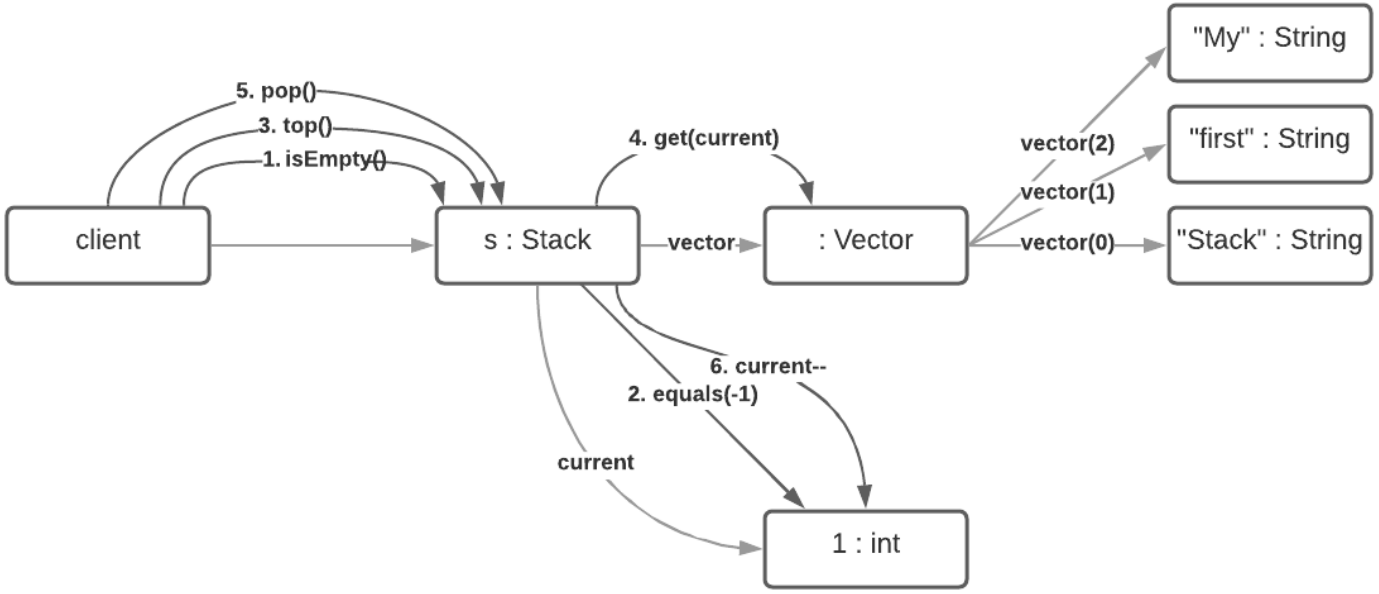


example

Architecture – Kruchten 4+1 View Model
Example 2

How do you transfer the stack scenario into the Logical View of Kruchten's View Model?

Reading and emptying the Stack



example

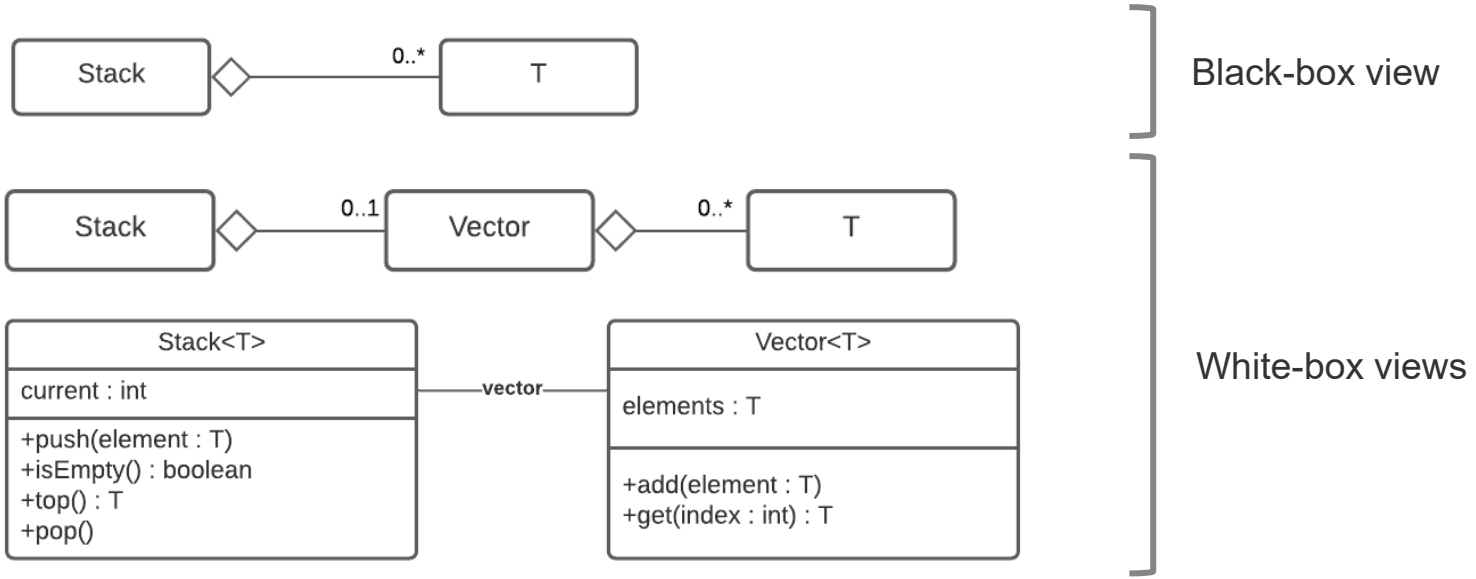
Architecture – Kruchten 4+1 View Model
Example 2

How do you represent the Stack in the development view?

example

Architecture – Kruchten 4+1 View Model
Example 2

How do you represent the Stack in the development view?



example

Architecture – Kruchten 4+1 View Model
Example 2

Finally, how do you code your Stack model in Java?

Architecture – Kruchten 4+1 View Model
Example 2

Finally, how do you code your Stack model in Java?

```
public class Stack<T> {
	private List<T> vector = new Vector<T>();
	private int current = -1;

	public void push(T element) {
		vector.add(element);
		current++;
	}
	public boolean isEmpty() {
		return current == -1;
	}
	public T top() {
		T result = null;
		if(!isEmpty()) {
			result = vector.get(current);
		}
		return result;
	}
	public void pop() {
		current--;
	}
}
```

```
public class Client {
	public static void main(String[] args) {
		Stack<String> s = new Stack<String>();

		s.push("Stack");
		s.push("first");
		s.push("My");

		while(!s.isEmpty()) {
			String top = s.top();
			System.out.println(top + " ");
			s.pop();
		}
	}
}
```

Lecture Agenda

- Architecture View Model
- Kruchten 4+1 View Model
- Development View to Logical View

Development View to Logical View
Motivation

When designing and programming software, you edit components that are mainly visible in the development View. However, the **actual goal is the effect the system causes when it is executed**.

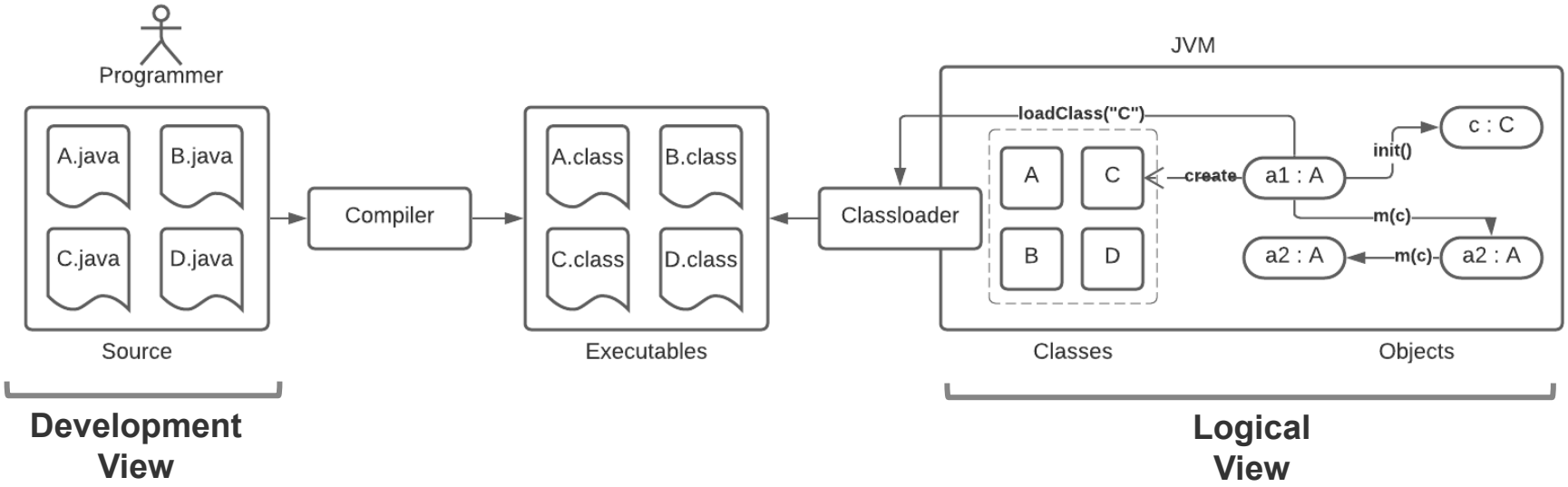
In the object oriented paradigm this **effect is produced by the interaction of objects**. Kruchten's **logical view** is the vehicle supporting a respective perspective.

Software structures are to be modelled in close relation to those of the subject matter domain.

Goal is a better and more understandable representation of the inner logic of software systems. In principle terms this is stated by the ***low representational gap* principle**.

Development View to Logical View
Overview

The path from programming to the effect of executed software comprises several intermediate steps, which we are not always fully conscious of. Therefore it is worthwhile to mind the fundamental procedure (i.e., from the development view to the logical view).



Development View to Logical View
Overview

A **software system aims at causing an effect** (e.g., in the computer's memory, at the user interface level, or at interfaces to other software).

This effect is caused **by the behavior of objects** in memory and their interaction.

The computer (i.e., java virtual machine) has embedded **a factory for such objects**. It consists of the *classLoader* and class descriptors with methods for creating new objects of a respective class (e.g., *new*, *forName*).

The object factory **uses construction blueprints (i.e., *.class* files) in factoring objects**. The **compiler creates *.class* files from the source files** (i.e., *.java* files) developed during programming.

To be able to estimate or consciously control the effect caused at runtime, you must understand the run time structures and create the design-time structures (classes) in such a way create that the necessary object structures can be produced.

You therefore **begin the design of systems with the *logical view***.

Low Representational Gap
Overview

The gap between different representations of real-world objects, models and software should be as small as possible. The object oriented paradigm uses object-oriented models and finally object-oriented programs in order to achieve a gap as small as possible.

Real World

Four wheels are mounted on the car: front left, front right, rear left, rear right.

Object Model

?

Class Model

?

Code Model

?

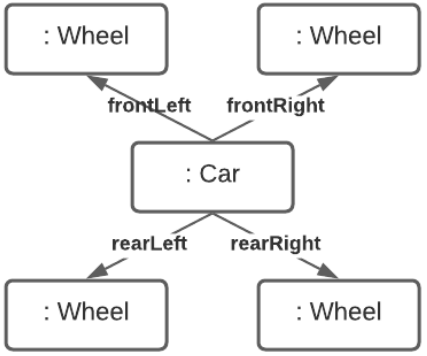
Low Representational Gap
Overview

The gap between different representations of real-world objects, models and software should be as small as possible. The object oriented paradigm uses object-oriented models and finally object-oriented programs in order to achieve a gap as small as possible.

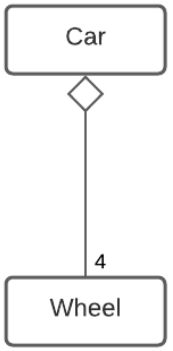
Real World

Four wheels are mounted on the car: front left, front right, rear left, rear right.

Object Model



Class Model



Code Model

```
public class Car {
   Wheel frontLeft, frontRight;
   Wheel rearLeft, rearRight;
}
public class Wheel { }
```

Low Representational Gap
Overview

The LRG principle requires the **representation of the real world in the form of an object-oriented model** (i.e., create a model of the real world).

The second step is to try to **represent the resulting model 1:1 in software**. This cannot always be realized down to the last detail, but the aim serves as a reasonable heuristic. Ideally, this is a process that can be carried out methodically and reproducibly.

Following the LRG principle …

- the objects, their data or states and behavior correspond as closely as possible to real world objects.
- between real objects, data and program structures and finally code there is only an as small as possible gap.
- object and class structures and the code can be understood more systematically and efficiently. This helps to avoid errors and makes it easier for other people to familiarize themselves with the solution later on.

Low Representational Gap
Overview

In order to adopt the LRG principle, the **design must be guided primarily by the structures of the subject matter domain** and less by technical structures.

There are various alternative approaches, like entity-relationship modelling (**ERM**) in relational database design, object-oriented analysis with object-oriented design (**OOA, OOD**), domain-driven design (**DDD**), model-driven architecture (**MDA**).

All approaches have in common that you initially describe the real world, in which the software is to cause something, with a model. The structures of the software are then oriented to the structures of this model.

example

Low Representational Gap
Example

In this example, we consider the address management of a company, in which data of individual persons are stored.

A *person* can assume the following roles: *customer*, *supplier* and *employee*.

All persons share the same attributes: *name*, *first name*, *address* (i.e. street and number), *postal code* and *city*.

Depending on the role of a person, additional attributes are stored. For example, *customers* have an *annual turnover* (cumulative for the current year), *suppliers* have a *quality rating* from previous deliveries, and *employees* work for a *department* and have an *annual salary*.

example

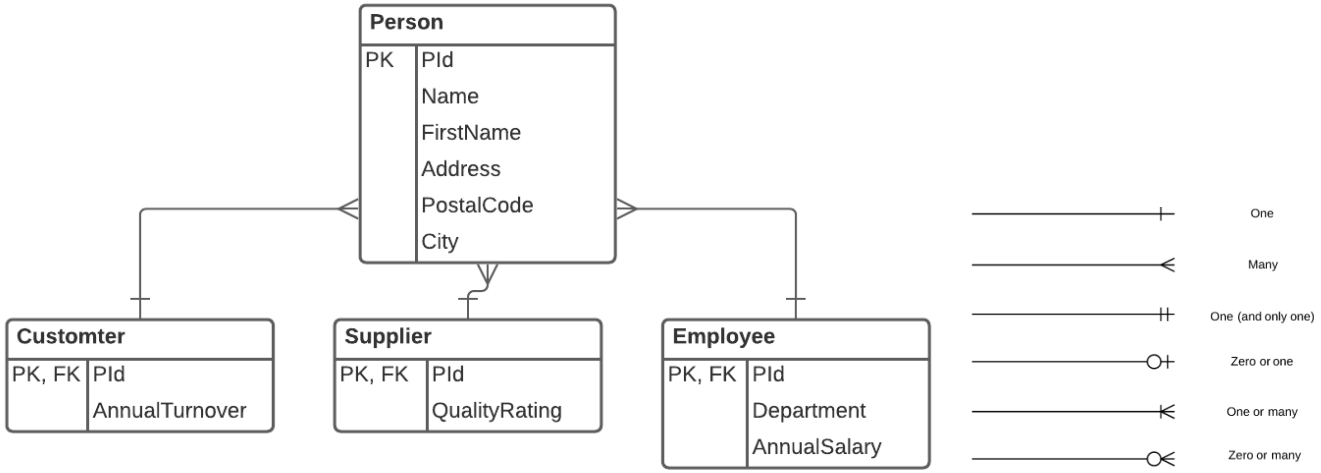
Low Representational Gap
Example

How are these entity types represented in a relational database in the development view?

example

Low Representational Gap
Example

How are these entity types represented in a relational database in the development view?



example

Low Representational Gap
Example

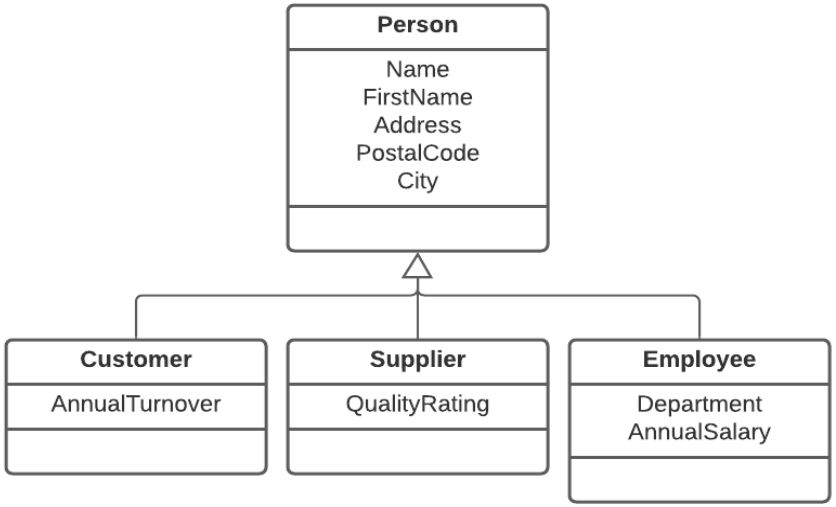
How are these entity types represented in an object oriented design in the development view?


example

Low Representational Gap
Example

How are these entity types represented in an object oriented design in the development view?

- While the use of primary and foreign keys is one of the means of expression of relational database design, the is-a relationship (subtyping or type inheritance) is a means of expression of object orientation.
- Such is-a relationships, expressed in relational databases with foreign keys, can be represented more directly between objects: A customer is a person.



example

Low Representational Gap
Example

In the Logical View, differences between the two approaches become evident, especially with regard to when is-a relationships are established with concrete data.

The following scenario makes the differences visible:

1. customer Luis Müller, Bergweg 9, 8015 Exemplikon has an annual turnover of 284'295.
2. supplier Katja Maier, Hauptstrasse 525, 4711 Strasswil has a quality rating of 5.
3. employee Jonas Huber, Seitenstrasse 98, 2113 Waldbach works in the accounting department for an annual salary of 87’654.
4. Luis Müller, already known as a customer, is additionally employed in the shipping department with an annual salary of annual salary of 78,323.

example

Low Representational Gap
Example

How are these entities represented in a relational database in the logical view?

Person

PId	Name	FirstName	Address	PostalCode	City

Customer

PId	AnnualTurnover

Supplier

PId	QualityRating

Employee

PId	Department	AnnualSalary

example

Low Representational Gap
Example

How are these entities represented in a relational database in the logical view?

Person

PId	Name	FirstName	Address	PostalCode	City
1	Müller	Luis	Bergweg 9	8019	Exemplikon
2	Maier	Katja	Hauptstrasse 525	4711	Strasswil
3	Huber	Jonas	Seitenstrasse 98	2113	Waldbach

Customer

PId	AnnualTurnover
1	284'295

Supplier

PId	QualityRating
2	5

Employee

PId	Department	AnnualSalary
1	shipping	78,323
3	accounting	87'654

example

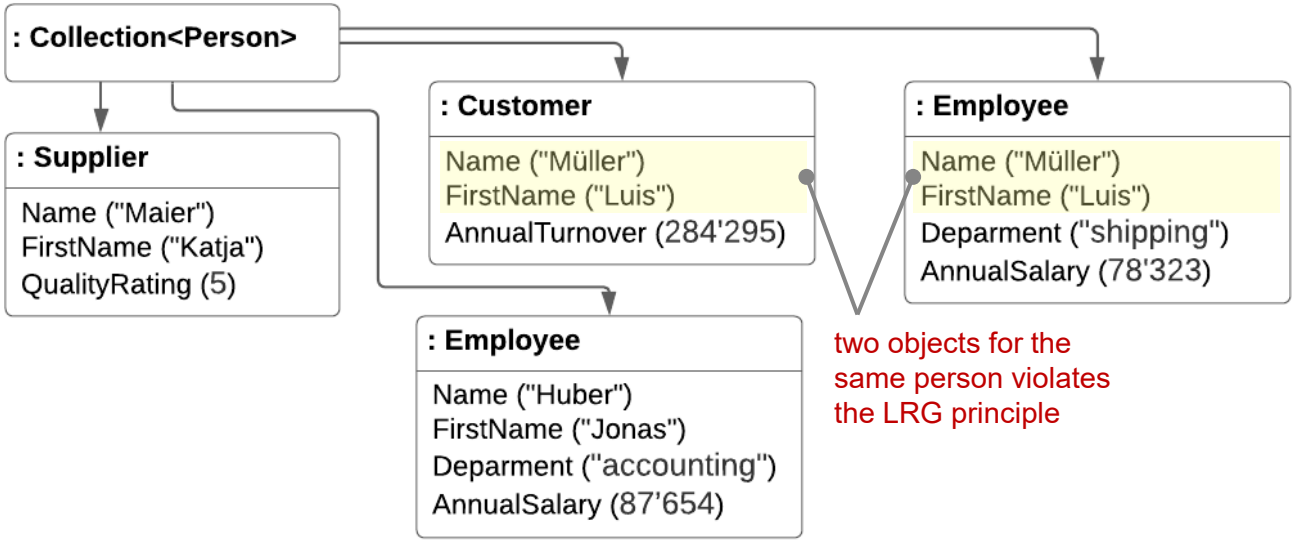
Low Representational Gap
Example

Now, how are the above entities represented in an object oriented design in the logical view?

example

Low Representational Gap
Example

Now, how are the above entities represented in an object oriented design in the logical view?



example

Low Representational Gap
Example

Compare the two designs (i.e., ERM and OOD). What are the differences regarding to the associations of people with roles?

example

Low Representational Gap
Example

Compare the two designs (i.e., ERM and OOD). What are the differences regarding to the associations of people with roles?

- The ERM allows one *person* to have multiple *roles* at the same time. For the OOD, the respective classes for the different role combinations would have to be developed first.
- While the ERM allows a *role* to be added to or removed from a *person* at any time, the OOD specifies the *role* for a *person* at instantiation time.
- The OOD guarantees that a *person* object represents at most one *role*.

example

Low Representational Gap
Example

What does an object-oriented design look like that offers similar flexibility in the logical view as the relational database design above?

Development view

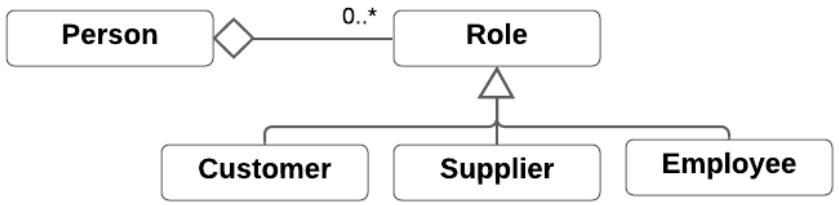
Logical view

example

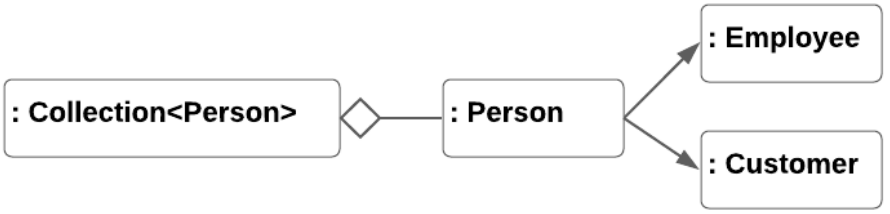
Low Representational Gap
Example

What does an object-oriented design look like that offers similar flexibility in the logical view as the relational database design above?

Development view



Logical view



Behaviour and Methods in the Logical View
Overview

In the previous slides, we followed a **data-driven design**. This means that the modelled entities and their relationships were merely derived from data structures.

Operations, performed by individual objects, **were completely disregarded**. This is despite the fact that the encapsulation of data and operations is a pivotal feature of object-oriented programming.

A simple design strategy is to provide (for each object) all operations that seem useful for manipulating the data encapsulated in the object. However, such a **design is entity-driven**. Decisions are made with respect to each individual entity (or more precisely, each entity type) in isolation.

This works as long as the operations are more or less trivial. If **more complex operations** are expected (e.g., performing complex calculations, or having to change a set of attributes together and consistently), it often makes more sense to be **guided by what other objects (in an interaction scenario) require**.

Behaviour and Methods in the Logical View
Overview

Therefore, start from **scenarios that consider the system as a whole** and describe the cooperation between objects that is necessary for them to jointly realize a given scenario.

Such design is **scenario-driven**. It only considers the cooperation between objects and not the internal realization of corresponding methods. Thus, it is a **top-down approach** that abstracts from the details, such as the internal implementation of the objects.

Once it is known what operations the objects must provide because they are called to implement the scenarios, one can directly **derive the publicly visible elements** (e.g., methods, parameters, expected outcomes) from these objects.

The internals of the objects or classes can be left to **detailed design** thanks to **information hiding**.

Development View
Motivation

A design oriented to the application domain in the paradigm of object orientation provides a blueprint in the **logical view** that respects the LRG principle.

The blueprint **contains objects and represents their cooperation** (i.e., mutual method calls of the objects) to realize the identified use cases. For this dynamic view, one usually uses sequence or communication diagrams.

Provided that a sufficient variety of use case realizations was elaborated, these **blueprints contain an architecture inherently in terms of interacting components** in the logical view.

The interactions between the objects in the form of operation calls are fixed which **limits the degrees of freedom in the realization of the individual objects**.

For the conversion into executable software one must now still produce the necessary programs (e.g., classes) for the individual objects – this is a job for Kruchten's development view.

Development View
Overview

Model-, domain- and object-oriented design methods aim at a LRG between the well recognizable structures of the real world and those of the software.

Depending on the complexity of the task, you can proceed in one direct or two successive steps. In the latter case, you start with the modelling of the real world in the course of object-oriented analysis (OOA) as a basis on which an object-oriented design (OOD) is developed.

In contrast to analysis, objects are taken into account in the design, which have no direct counterpart in the real world (and are therefore missing in the analysis), while nevertheless their implementation is considered necessary. Examples are technical or administrative concerns.

A goal of object-oriented analysis and design is to understand which objects produce the desired effect and how these objects respectively cooperate (logical view).

Development View
Overview

Such design consists of **static structure models** which indicate the required objects (and their classes) and their relationships. Second, it consists of **behaviouristic models** that show how these objects work together to realize the required scenarios.

The **object-oriented paradigm connects logical and development view via the concept of classes**. The algorithmic program logic is described with objects.

However, the **artifacts to be programmed are the classes** based on which objects are instantiated.

Methods an object calls on another or messages an object sends to another, must be publicly available and implemented by classes.

Relationships between objects require stored references (e.g., in instance variables).

Development View
Overview

For all simple objects which present themselves identically to the outside, a common class is sufficient.

For more **complex objects, on the other hand, a whole series of further objects and thus classes may be required** behind the façade of a generating class. However, these are not part of the architecture-relevant design.

Common aspects of classes for different objects can possibly be **delegated to super classes**, which are then differently extended.

Design decisions in the development view often reflect what you anticipate as future changes to the solution. Also, you may introduce components to be able to utilize polymorphism. Usually in such cases you need to define a suitable type hierarchy.

Development View
Overview

For the systematic development of an **object-oriented design in the development view, the steps below are performed**.

1. You create a list of all classes to be programmed, so that ...
   - each object can be instantiated by exactly one class
   - identically behaving objects are instantiated by a common class
   - each class instantiates at least one of the required objects
2. Each arrow arriving at an object (e.g., in a sequence diagram) represents the activation of a method. For these you provide a callable (public) method in the corresponding class. The result of this step are complete method signatures including *return value* and *parameters* as well as additional *information about method behaviour*. This means at this stage you also consider which information is passed between objects.
3. Classes defined in such a way can be independently programmed, before they are combined to shape the algorithmic structures realizing identified use cases.

Questions

Bibliography
Lecture

[ArchiMate® 2021]
Open Group, *ArchiMate*, 2021, https://www.opengroup.org/archimate-home

[Apostel 1960]
Apostel, Leo, Towards the formal study of models in the non-formal sciences, Synthese 12, 1960

[Kruchten 1995]
Kruchten, Philippe, *Architectural Blueprints – The '4+1' View Model of Software Architecture*, IEEE Software 12 (6), November 1995, pp. 42-50, http://dx.doi.org/10.1109/52.469759, https://www.researchgate.net/publication/220018231_The_41_View_Model_of_Architecture

[TOGAF® 2021]
Open Group, *The Open Group Architecture Framework*, 2021, https://www.opengroup.org/togaf

Bibliography
Lecture

[Vogel, Arnold et al 2011]
Vogel, Oliver; Arnold, Ingo; Chugtai, Arif; Kehrer, Timo, *Software Architecture: A Comprehensive Framework and Guide for Practitioners*, Springer Science and Business Media, Berlin Heidelberg, 2011

[Zachman 2021]
Zachman, Zachman; *the Zachman framework*, 2021, https://www.zachman.com/